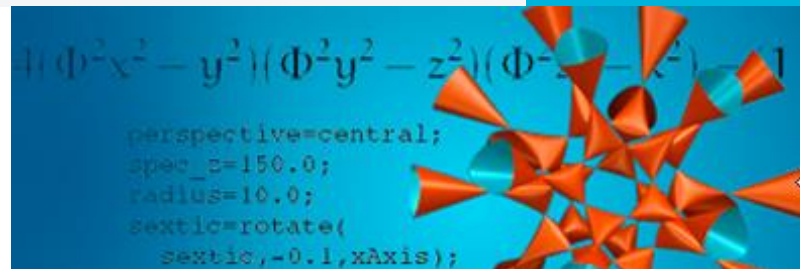


Theoretische Informatik: Grammatiken und Sprachen

Prof. Dr. Elmar Tischhauser



Inhalt

- Kontextfreie Grammatiken
- Ableitungen, Linksableitungen
- Sprachen zu einer Grammatik
- Äquivalenz
- Chomsky-Normalform
- Wortproblem, CYK
- Pumping Lemma für CF-Sprachen
- Kontextabhängige Sprachen
- Chomsky Grammatiken
- Chomsky-Hierarchie

Der zweistufige Aufbau von Sprachen

Aus Alphabet der Token wird der syntaktische Anteil der Sprache aufgebaut,

Σ = Menge der Token

L = Menge aller syntaktisch korrekten Sätze.

Auf dieser Stufe sind Sprachen i.A. nicht regulär. (Geschachtelte Strukturen kann man nicht regulär beschreiben - Klammersprache).

Für die Definition muss was neues her:

Grammatiken

Syntax - Semantik

- Grammatik beschreibt die **Syntax** (Struktur) von Sätzen, nicht deren **Semantik** (Bedeutung).
- Semantik hängt von Syntax ab, aber nicht jeder syntaktisch korrekte Satz muss eine Semantik haben.

Syntaktisch falscher Satz:

gestohlen der hat Gans Fuchs die

Syntaktisch richtiger, semantisch falscher Satz:

Die Gans hat den Fuchs gestohlen.

Syntaktisch und semantisch richtiger Satz:

Der Fuchs hat die Gans gestohlen.



Syntax - Semantik in PASCAL

Syntaktisch falsches Programm:

```
Test Program.  
  i for 1 do n ; to n+1 := n
```

Syntaktisch richtiges, semantisch falsches Programm:

```
Program Test ;  
Var i : Real;  
Begin  
  For i := 1 To n Do i := n-1  
End.
```

Syntaktisch und semantisch korrektes Programm:

```
Program Test ;  
Var i,n : Integer;  
Begin  
  For i := 1 To 50 Do n := n+i  
End.
```

Sprachen und Grammatiken

Eine **Grammatik** ist eine Menge von Regeln mit deren Hilfe „syntaktisch korrekte“ Worte (Sätze) der Sprache konstruiert werden können.

Erzeugen (ableiten)

Mit Hilfe der Grammatik kann man Sätze einer Sprache erzeugen. Die Menge dieser Sätze bilden die zur Grammatik G gehörende Sprache, $L(G)$.

Zerlegen (erkennen, entscheiden)

Ist ein Satz gegeben, so möchte man feststellen, ob er zur Sprache $L(G)$ gehört oder nicht. Dabei wird auch die Struktur des Satzes erkannt.

Beispiel einer Grammatik

Java

Stmt $\text{:= while (Expr) Stmt} \mid \text{if (Expr) Stmt else Stmt}$
 $\mid \text{id = Expr ;} \mid \{ \text{Stmts} \} \dots$

Expr $\text{:= Expr op Expr} \mid (\text{Expr}) \mid \text{num} \mid \text{id} \dots$

Stmts $\text{:= Stmt Stmts} \mid \varepsilon$
etc.

Bestandteile:

Token	: while, (,), if, else, id, =, ;, {, }, op, num, id
Variablen	: Stmt, Expr, Stmts,
Satzformen	: (Expr) , id = Expr ; , ε ,

Die Zeichen

:= (definiert als)
 $\mid$ (Alternative)

sind Sonderzeichen für Grammatikdefinitionen

Kontextfreie Grammatik

Gegeben:

- T - ein Alphabet (Terminale, Token)
- V - eine Menge von Variablen (Nonterminale)
- $S \in V$ - Startsymbol

mit $T \cap V = \emptyset$

Eine **Kontextfreie Grammatik** ist ein Quadrupel $G = (V, T, P, S)$
mit $T \cap V = \emptyset$,

$S \in V$ und $P \subseteq V \times (T \cup V)^*$

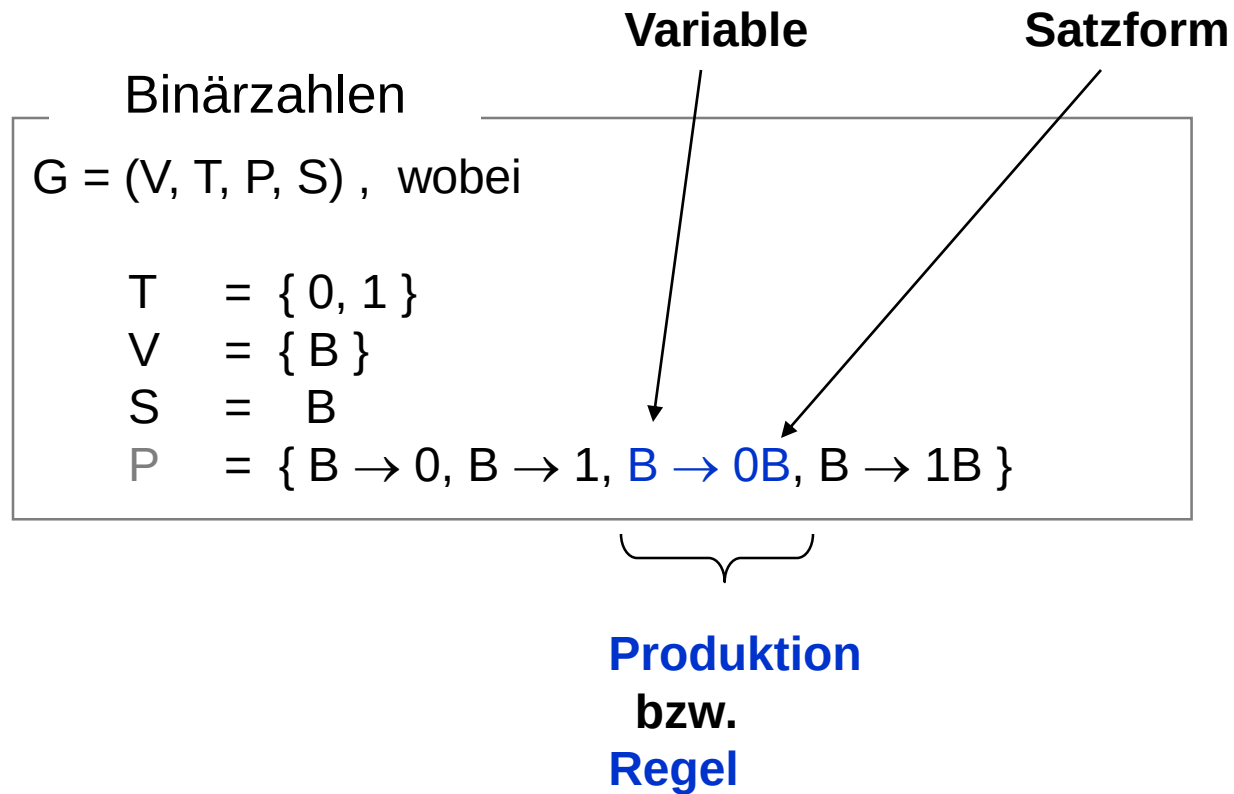
Begriffe

Satzform : Element α von $(V \cup T)^*$

Produktion : Jedes $(A, \alpha) \in P$. Notation: $A \rightarrow \alpha$

Startsymbol : S.

CF-Grammatik – Beispiel 1



CF-Grammatik – Beispiel 2

Ausdrücke

$G = (V, T, P, S)$, wobei

$T = \{ \text{id, num, +, *, }, (\}$

$V = \{ \text{Term, Factor, Expr} \}$

$S = \text{Expr}$

$P = \{$
 $\text{Expr} \rightarrow \text{Term} , \text{Expr} \rightarrow \text{Expr} + \text{Term},$
 $\text{Term} \rightarrow \text{Factor} , \text{Term} \rightarrow \text{Term} * \text{Factor},$
 $\text{Factor} \rightarrow \text{id}, \text{Factor} \rightarrow \text{num},$
 $\text{Factor} \rightarrow (\text{Expr})$
 $\}$

Konventionen

- **Variablen** : Alle Symbole für die es mind. eine Produktion gibt
- **Terminale** : alle anderen Symbole
- **Startsymbol** :
 - Default: Variable der ersten Produktion.
- Produktionen mit **gleicher linker Seite** zusammenfassen.
 - Rechte Seiten durch | trennen.

Ausdrücke

$G = (V, T, P, S)$, wobei

$T = \{ \text{id, num, +, *, }, (\}$

$V = \{ \text{Term, Factor, Expr} \}$

$S = \text{Expr}$

$P = \{ \text{Expr} \rightarrow \text{Term}, \text{Expr} \rightarrow \text{Expr} + \text{Term}, \\ \text{Term} \rightarrow \text{Factor}, \text{Term} \rightarrow \text{Term} * \text{Factor}, \\ \text{Factor} \rightarrow \text{id}, \text{Factor} \rightarrow \text{num}, \\ \text{Factor} \rightarrow (\text{Expr}) \}$

Das vorige Beispiel kann man dann schreiben als :

Ausdrücke

$\text{Expr} \rightarrow \text{Expr} + \text{Term} \\ | \text{Term}$

$\text{Term} \rightarrow \text{Term} * \text{Factor} \\ | \text{Factor}$

$\text{Factor} \rightarrow (\text{Expr}) \\ | \text{id} | \text{num}$

Ersetzung - Ableitung

Eine Produktion $A \rightarrow \beta$ erlaubt, in einer Satzform $\alpha A \gamma$ die Variable A durch β zu ersetzen. Resultat ist die Satzform $\alpha \beta \gamma$.

Eine Produktion:

$$A \rightarrow \beta$$

Eine Ersetzung:

$\alpha A \gamma$

$\Rightarrow$

$\alpha \beta \gamma$

$\sigma \Rightarrow^* \tau$ falls es $\alpha_1, \dots, \alpha_n$ gibt mit

$\sigma = \alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \alpha_n = \tau$

Ableitung

Beispiel einer Ableitung

Arithmetik

$$\text{Expr} \rightarrow \text{Expr} + \text{Term} \\ | \text{Term}$$
$$\text{Term} \rightarrow \text{Term} * \text{Factor} \\ | \text{Factor}$$
$$\text{Factor} \rightarrow (\text{Expr}) \\ | \text{id} \mid \text{num}$$

Es gilt also:

$$\text{Expr} \Rightarrow^* \text{id} * (\text{num} + \text{id})$$

Eine Ableitung:

$$\text{Expr} \Rightarrow \text{Term}$$
$$\begin{aligned} &\Rightarrow \text{Term} * \text{Factor} \\ &\Rightarrow \text{Factor} * \text{Factor} \\ &\Rightarrow \text{id} * \text{Factor} \\ &\Rightarrow \text{id} * (\text{Expr}) \\ &\Rightarrow \text{id} * (\text{Expr} + \text{Term}) \\ &\Rightarrow \text{id} * (\text{Term} + \text{Term}) \\ &\Rightarrow \text{id} * (\text{Factor} + \text{Term}) \\ &\Rightarrow \text{id} * (\text{num} + \text{Term}) \\ &\Rightarrow \text{id} * (\text{num} + \text{Factor}) \\ &\Rightarrow \text{id} * (\text{num} + \text{id}) \end{aligned}$$

Eine Links-Ableitung

Arithmetik

$$\text{Expr} \rightarrow \text{Expr} + \text{Term} \\ | \text{Term}$$
$$\text{Term} \rightarrow \text{Term} * \text{Factor} \\ | \text{Factor}$$
$$\text{Factor} \rightarrow (\text{Expr}) \\ | \text{id} \mid \text{num}$$

Jedes Wort $w \in L(G)$ kann man auch durch eine Linksableitung gewinnen

Bei einer **Links-Ableitung** wird immer das linkeste Nonterminal ersetzt

Eine Links-Ableitung:

$\text{Expr} \Rightarrow \text{Term}$

- $\Rightarrow \text{Term} * \text{Factor}$
- $\Rightarrow \text{Factor} * \text{Factor}$
- $\Rightarrow \text{id} * \text{Factor}$
- $\Rightarrow \text{id} * (\text{Expr})$
- $\Rightarrow \text{id} * (\text{Expr} + \text{Term})$
- $\Rightarrow \text{id} * (\text{Term} + \text{Term})$
- $\Rightarrow \text{id} * (\text{Factor} + \text{Term})$
- $\Rightarrow \text{id} * (\text{num} + \text{Term})$
- $\Rightarrow \text{id} * (\text{num} + \text{Factor})$
- $\Rightarrow \text{id} * (\text{num} + \text{id})$

Links-Ableitung eines anderen Wortes

Arithmetik

$$\text{Expr} \rightarrow \text{Expr} + \text{Term} \\ | \text{Term}$$
$$\text{Term} \rightarrow \text{Term} * \text{Factor} \\ | \text{Factor}$$
$$\text{Factor} \rightarrow (\text{Expr}) \\ | \text{id} \mid \text{num}$$

Auswahl anderer Regeln
erzeugt i.A. andere Worte
 $w \in L(G)$

Bei einer **Links-Ableitung** wird immer das linkeste Nonterminal ersetzt

Eine Links-Ableitung unter Verwendung anderer Regeln:

$\text{Expr} \Rightarrow \text{Expr} + \text{Term}$
 $\Rightarrow \text{Term} + \text{Term}$
 $\Rightarrow \text{Factor} + \text{Term}$
 $\Rightarrow \text{num} + \text{Term}$
 $\Rightarrow \text{num} + \text{Factor}$
 $\Rightarrow \text{num} + (\text{Expr})$
 $\Rightarrow \text{num} + (\text{Term})$
 $\Rightarrow \text{num} + (\text{Term} * \text{Factor})$
 $\Rightarrow \text{num} + (\text{Factor} * \text{Factor})$
 $\Rightarrow \text{num} + (\text{id} * \text{Factor})$
 $\Rightarrow \text{num} + (\text{id} * \text{num})$

Ableitung als Baum

Arithmetik

$\text{Expr} \rightarrow \text{Expr} + \text{Term}$
| Term

$\text{Term} \rightarrow \text{Term} * \text{Factor}$
| Factor

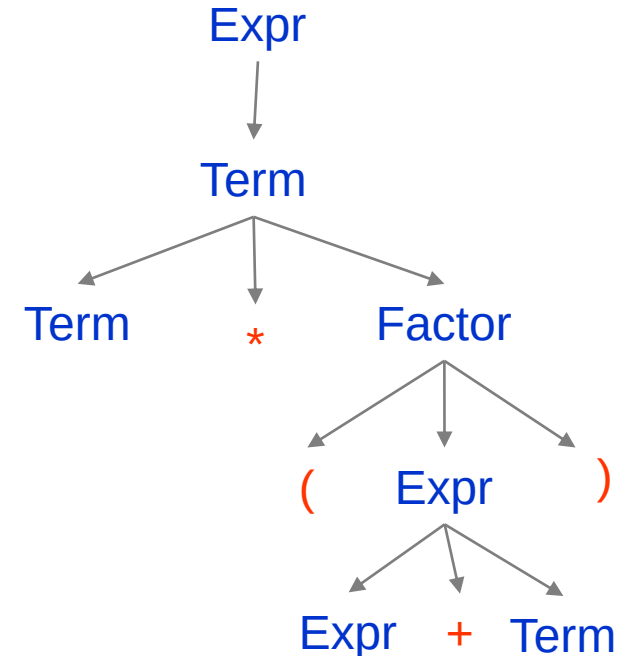
$\text{Factor} \rightarrow (\text{Expr})$
| id | num

$\text{Expr} \Rightarrow \text{Term}$
 $\Rightarrow \text{Term} * \text{Factor}$
 $\Rightarrow \text{Term} * (\text{Expr})$
 $\Rightarrow \text{Term} * (\text{Expr} + \text{Term})$

angefangene Ableitung

Ableitungen in Baumnotation:

- **Wurzel:** Startsymbol
- Anwendung einer Regel $A \rightarrow \alpha$
 - Knotenwachstum.
- Symbole aus α werden Kindknoten



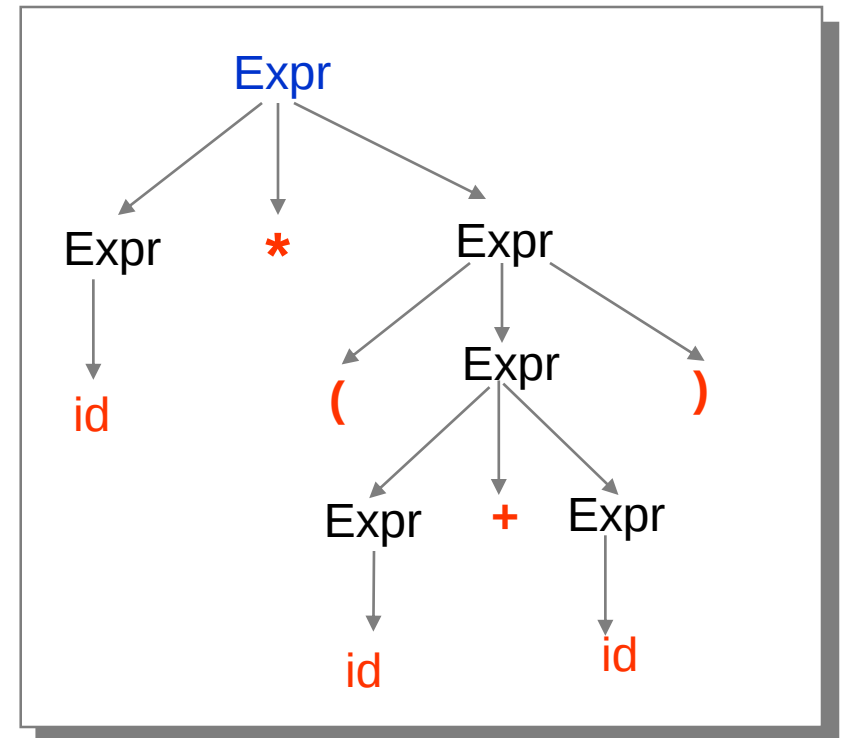
entsprechender Ableitungsbaum

Fertiger Ableitungsbaum

G

$\text{Expr} \rightarrow \text{Expr} + \text{Expr}$
 $\quad \mid \text{Expr} * \text{Expr}$
 $\quad \mid (\text{Expr})$
 $\quad \mid \text{id}$

Wurzel: Startsymbol
innere Knoten: Variablen
Söhne von A : Symbole von α ,
für $A \rightarrow \alpha \in P$
Blätter: Terminalsymbole



repräsentiert Ableitung
 $\text{Expr} \Rightarrow^* \text{id} * (\text{id} + \text{id})$

Blätter des Baumes ergeben von links nach rechts das abgeleitete Wort.

Grammar-Editor (C. Burch)

Grammar Editor

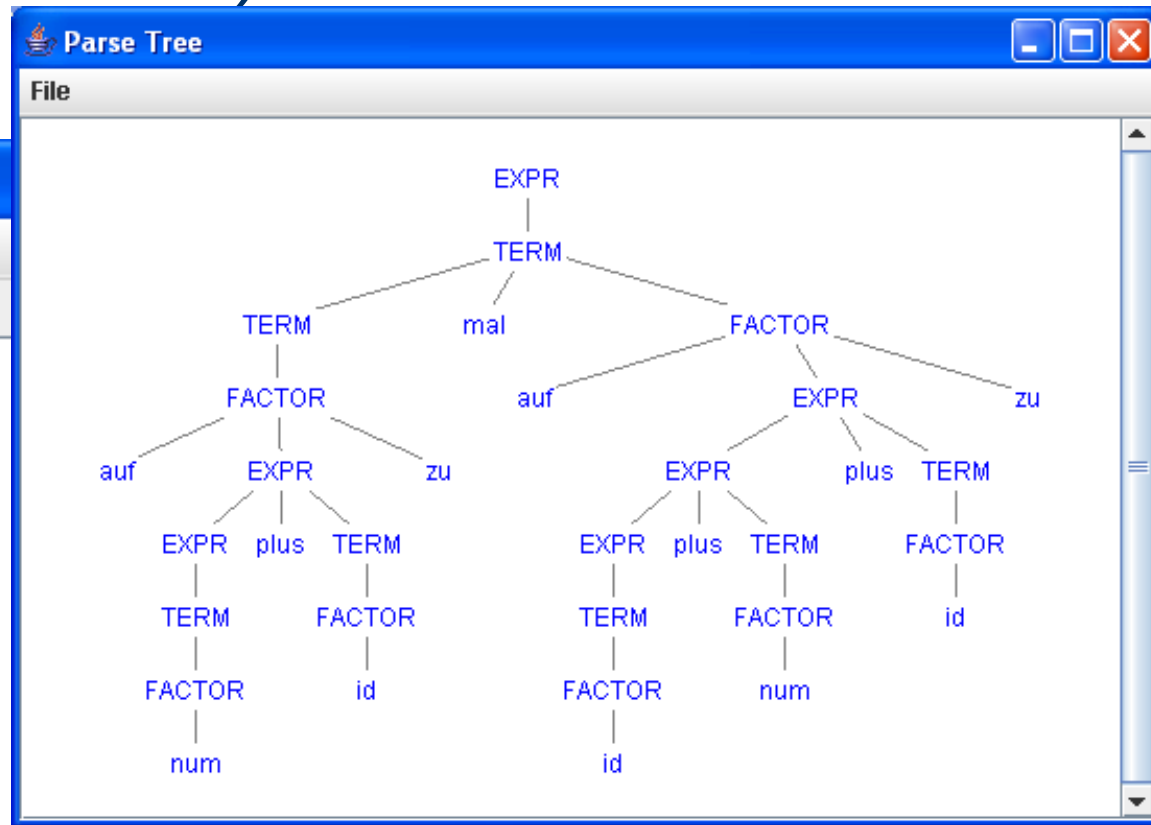
File Edit Options Help

Grammar:

```
# Ausdrucks-Grammatik  
EXPR -> TERM  
| EXPR plus TERM  
TERM -> FACTOR  
| TERM mal FACTOR  
FACTOR -> num  
| id  
| auf EXPR zu
```

Text: auf num plus id zu mal auf id plus num plus id zu

Parse Generate Clear



nettes Spielzeug-Tool

erzeugt Worte einer Grammatik

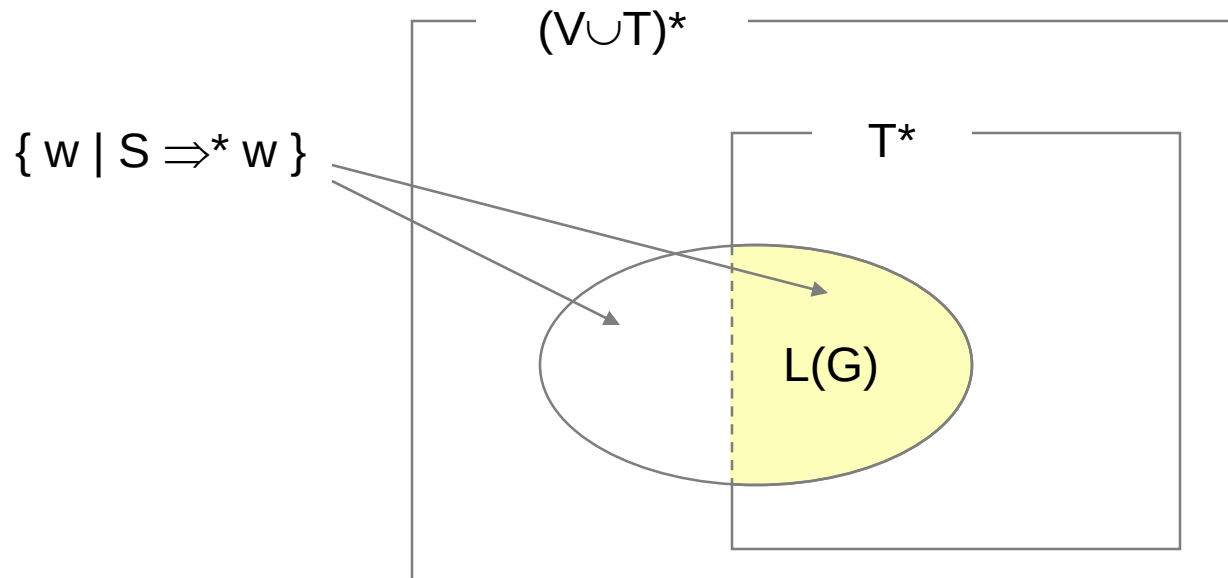
generiert und zeigt Ableitungsbaum

Die Sprache zu einer Grammatik

Ist $G = (V, T, P, S)$ eine Grammatik, so ist

$$L(G) = \{ w \in T^* \mid S \Rightarrow^* w \}$$

die von G erzeugte Sprache.



Kontextfreie Sprachen – Beispiel 3

- Eine Sprache $L \subseteq T^*$ heißt **kontextfrei**, falls es eine kontextfreie Grammatik G gibt, mit $L = L(G)$.

Beispiel:

$L_{anbn} = \{ a^n b^n \mid n \geq 0 \}$ ist kontextfrei:

$$\begin{array}{c} G \\ \hline S \rightarrow a S b \\ \quad \mid \varepsilon \end{array}$$

Muss man das beweisen? Kaum, denn es ist ziemlich offensichtlich. Aber wir können daran üben, wie man einen Beweis macht. Also:

1. $L_{anbn} \subseteq L(G)$:

Induktionsanfang $n=0$: klar, denn $a^0 b^0 = \varepsilon$ und $S \Rightarrow^* \varepsilon \in T^*$

Ind. Schritt:

Sei $a^k b^k \in L(G)$ schon gezeigt, dann gibt es eine Herleitung $S \Rightarrow^* a^k b^k$.

... und jetzt? $a^{k+1} b^{k+1}$? Tja – doch nicht so einfach?!

Kontextfreie Sprachen – Beispiel 3

- Eine Sprache $L \subseteq T^*$ heißt **kontextfrei**, falls es eine kontextfreie Grammatik G gibt, mit $L = L(G)$.

Beispiel:

$L_{anbn} = \{ a^n b^n \mid n \geq 0 \}$ ist kontextfrei:

$$\begin{array}{c} G \\ \hline S \rightarrow a S b \\ \quad \mid \varepsilon \end{array}$$

$$L_{anbn} = L(G).$$

Die Behauptung folgt aus der stärkeren Aussage:

Für jede Satzform $\alpha \in \{S, a, b\}^*$: $S \Rightarrow^* \alpha$ gdw. $\alpha = a^n S b^n \vee \alpha = a^n b^n$

Beweis: Sei $F = \{a^n S b^n \mid n \geq 0\} \cup \{a^n b^n \mid n \geq 0\}$. Dann gilt

$S \in F$ und

$\sigma \in F, \sigma \Rightarrow \tau$ impliziert $\tau \in F$

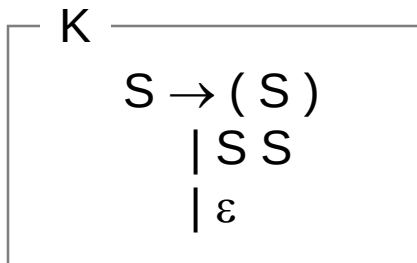
Daher: $S \Rightarrow^* \alpha$ impliziert $\alpha \in F$. Aus $w \in L(G)$ folgt also $w \in L_{anbn}$. Damit $L(G) \subseteq L_{anbn}$.

Umgekehrt: Für jedes n gilt $S \Rightarrow^* a^n S b^n$ (Induktion). Folglich auch $S \Rightarrow^* a^n S b^n \Rightarrow a^n b^n$.

Kontextfreie Sprachen – Beispiel 4

- Eine Sprache $L \subseteq T^*$ heißt **kontextfrei**, falls es eine kontextfreie Grammatik G gibt, mit $L = L(G)$.

Die Kammersprache ist kontextfrei:



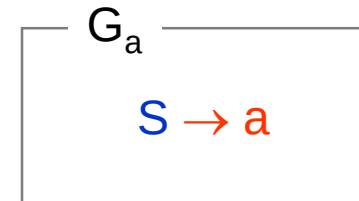
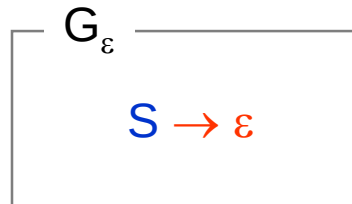
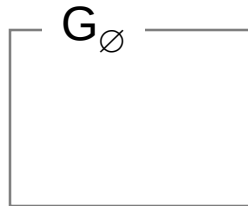
Beispiel einer Herleitung:

$S \Rightarrow (S)$
 $\Rightarrow (SS)$
 $\Rightarrow ((S)S)$
 $\Rightarrow (((S))S)$
 $\Rightarrow ((())(S))$
 $\Rightarrow (((()))()$

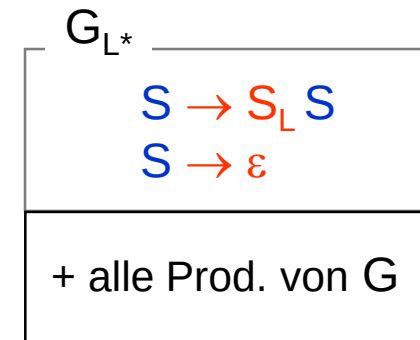
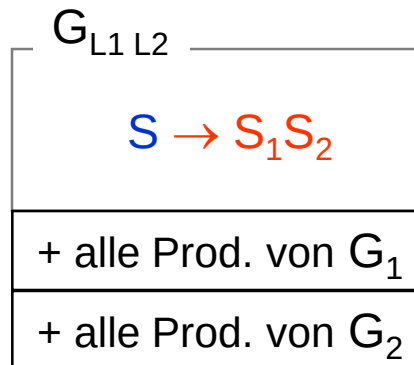
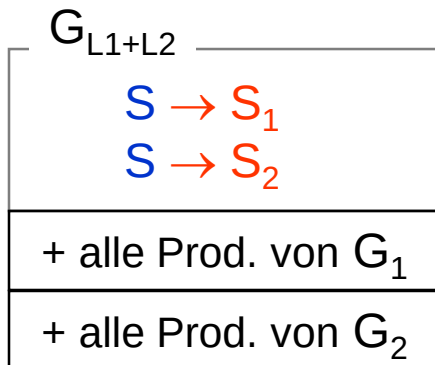
Hier ist nichts zu beweisen, weil wir die Kammersprache nie definiert haben.
Im Gegenteil, wir nehmen die obige Grammatik als **Definition für die Kammersprache**.

Jede reguläre Sprache ist kontextfrei

Sei Σ gegeben. Die Basissprachen :



Seien L, L_1, L_2 durch Grammatiken G, G_1, G_2 mit Startsymbolen S_L, S_1 , bzw. S_2 gegeben:



Sprachen zu einer Grammatik

Welche Sprache gehört zu folgender Grammatik?

$$\begin{array}{l} \text{--- } G \text{ ---} \\ S \rightarrow aB \mid bA \mid \varepsilon \\ A \rightarrow aS \mid bAA \\ B \rightarrow bS \mid aBB \end{array}$$

Vermutung: $L(G) = \{ w \in \{a,b\}^* \mid \#(a,w) = \#(b,w) \}$,
wobei

$\#(a,w)$ = Anzahl der a's in w.

Seien L_A , L_B , L_S die Sprachen, die entstehen, wenn man in G das entsprechende Nonterminal als Startsymbol nimmt. Wir zeigen die stärkere Behauptung:

$$L_S = \{ w \in \{a,b\}^* \mid \#(a,w) = \#(b,w) \}$$

$$L_A = \{ w \in \{a,b\}^* \mid \#(a,w) = \#(b,w) + 1 \}$$

$$L_B = \{ w \in \{a,b\}^* \mid \#(a,w) = \#(b,w) - 1 \}$$

Simultaner Beweis - Teil 1

G

$S \rightarrow aB \mid bA \mid \varepsilon$

$A \rightarrow aS \mid bAA$

$B \rightarrow bS \mid aBB$

$$L_S \subseteq \{ w \in \{a,b\}^* \mid \#(a,w) = \#(b,w) \}$$

$$L_A \subseteq \{ w \in \{a,b\}^* \mid \#(a,w) = \#(b,w) + 1 \}$$

$$L_B \subseteq \{ w \in \{a,b\}^* \mid \#(a,w) = \#(b,w) - 1 \}$$

Behauptung : Für jede Satzform α gilt:

$$S \Rightarrow^* \sigma \text{ impliziert } \#(a, \sigma) + \#(A, \sigma) = \#(b, \sigma) + \#(B, \sigma).$$

Induktion über Länge der Ableitung $S \Rightarrow^* \sigma$:

Gilt für $\sigma = S$ (Ableitung der Länge 0).

Jede Regelanwendung erhält die Eigenschaft (einzeln nachprüfen)

exemplarisch: $A \rightarrow bAA$

$$\alpha A \beta \Rightarrow \alpha bAA \beta$$

// links verschwindet ein A

// rechts kommen 2 A's und ein b hinzu.

Für $w \in L_S$ folgt also:

$$S \Rightarrow^* w \in \{a,b\}^*, \text{ somit } \#(a,w) = \#(b,w),$$

$$\text{also } L_S \subseteq \{ w \in \{a,b\}^* \mid \#(a,w) = \#(b,w) \}.$$

q.e.d.

Analog zeigt man die Behauptungen für L_A und L_B .

Simultaner Beweis - Teil 2, Fall 1

G

$S \rightarrow aB \mid bA \mid \varepsilon$

$A \rightarrow aS \mid bAA$

$B \rightarrow bS \mid aBB$

$L_S \supseteq \{ w \in \{a,b\}^* \mid \#(a,w) = \#(b,w) \} =: L_0$

$L_A \supseteq \{ w \in \{a,b\}^* \mid \#(a,w) = \#(b,w) + 1 \} =: L_{+a}$

$L_B \supseteq \{ w \in \{a,b\}^* \mid \#(a,w) = \#(b,w) - 1 \} =: L_{+b}$

Behauptung:

w aus einer der rechten Seiten, dann auch aus der entsprechenden linken.

Induktion über die Länge von w :

$w = \varepsilon$: Wenn ε aus einer der rechten Seiten ist, dann aus L_0 . In der Tat gilt $\varepsilon \in L_S$.

$w \neq \varepsilon$: Beispiel: $w \in L_{+a}$:

1. Fall: $w = a.v$:
 $v \in L_0$
also $v \in L_S$
also $S \Rightarrow^* v$
 $A \Rightarrow aS \Rightarrow^* a.v$
 $A \Rightarrow^* w$
 $w \in L_A$

// ($w \in L_0$, $w \in L_{+b}$ analog)

// Def. L_{+a} , L_0

// Ind. hypothese (v kürzer als w)

// Def L_S

// Erste Regel für A

// $\Rightarrow$ transitiv, $w = a.v$

// Def. L_A

Simultaner Beweis - Teil 2, Fall 2

G

$S \rightarrow aB \mid bA \mid \varepsilon$
 $A \rightarrow aS \mid bAA$
 $B \rightarrow bS \mid aBB$

$L_S \supseteq \{ w \in \{a,b\}^* \mid \#(a,w) = \#(b,w) \} =: L_0$
 $L_A \supseteq \{ w \in \{a,b\}^* \mid \#(a,w) = \#(b,w) + 1 \} =: L_{+a}$
 $L_B \supseteq \{ w \in \{a,b\}^* \mid \#(a,w) = \#(b,w) - 1 \} =: L_{+b}$

Behauptung:

w aus einer der rechten Seiten, dann auch aus der entsprechenden linken.

Induktion über die Länge von w :

$w = \varepsilon$: Wenn ε aus einer der rechten Seiten ist, dann aus L_0 . In der Tat gilt $\varepsilon \in L_S$.

$w \neq \varepsilon$: Beispiel: $w \in L_{+a}$:

// ($w \in L_0$, $w \in L_{+b}$ analog)

2. Fall: $w = b.v$: $\#(a,v) = \#(b,v) + 2$

// klar

$v = st$ mit $s, t \in L_{+a}$

// v kann so zerlegt werden

also $s, t \in L_A$

// Ind. hypothese (s, t kürzer als w)

also $A \Rightarrow^* s$, $A \Rightarrow^* t$

// Def L_A

$A \Rightarrow bAA \Rightarrow^* bsA \Rightarrow^* bst$

// Zweite Regel für A

$A \Rightarrow^* w$

// $\Rightarrow$ transitiv, $w = b.st$

$w \in L_A$

// Def. L_A

Äquivalenz

Grammatiken G_1 und G_2 heißen äquivalent, falls $L(G_1) = L(G_2)$.

$$\begin{array}{c} \mathbf{G_1} \\ \hline \text{Sum} \rightarrow \text{Sum} + \text{Sum} \\ | \quad \text{id} \end{array}$$

$$\begin{array}{c} \mathbf{G_2} \\ \hline \text{Sum} \rightarrow \text{Sum} + \text{id} \\ | \quad \text{id} \end{array}$$

G_1 und G_2 sind äquivalent:

$$L(G_1) = \text{id} ('+' \text{id})^* = L(G_2)$$



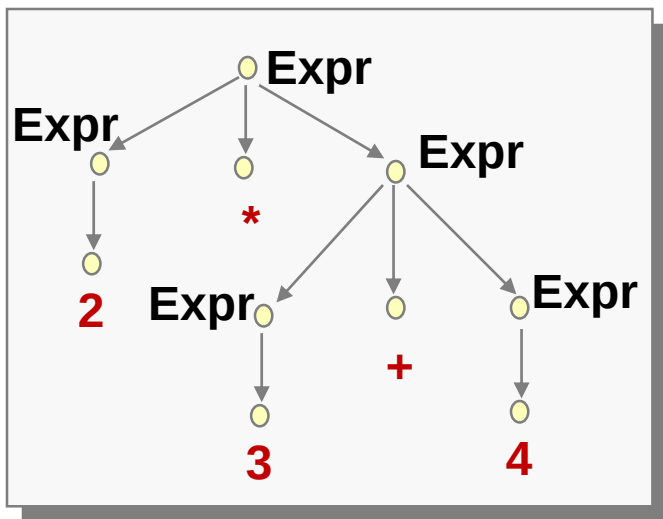
G. Frege

Bedeutung - Semantik

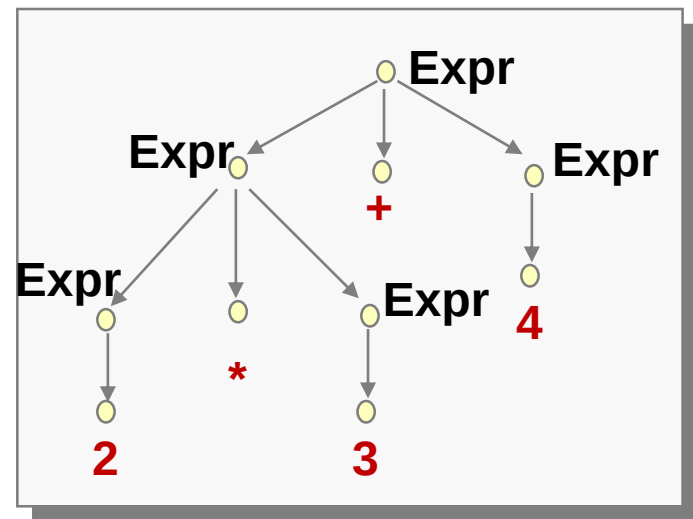
Denotationelles Prinzip (Frege): Die Bedeutung des Ganzen ist eine Funktion der Bedeutung der Teile.

Herleitungsbaum gibt Aufbau und damit die intendierte Bedeutung des Satzes eindeutig wieder. Lineare Schreibweise als String ist oft nicht eindeutig:

$$2 * 3 + 4 = ?$$



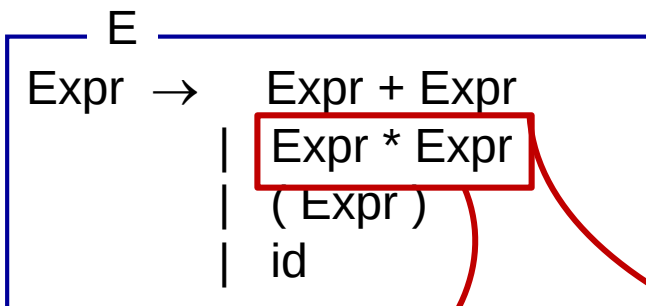
14



10

Mehrdeutigkeit

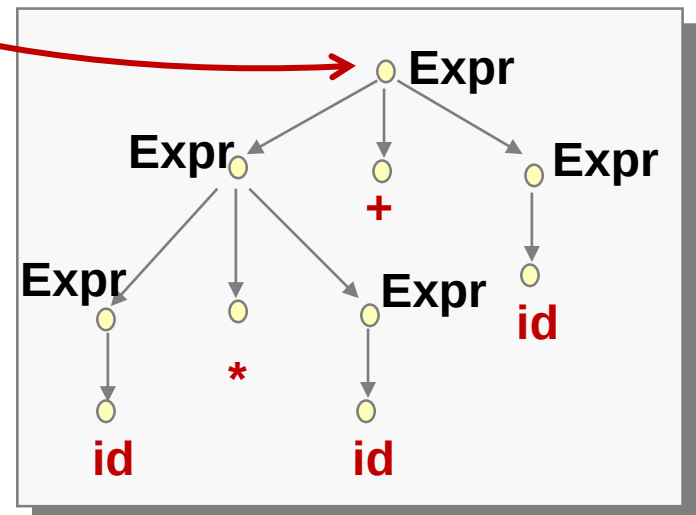
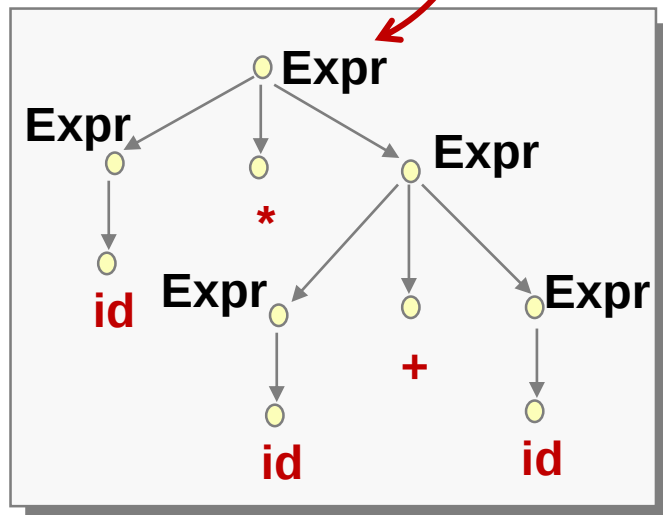
Besitzt $L(G)$ ein Wort w , zu dem es mehrere Herleitungsbäume gibt, so heißt G mehrdeutig.



Diese Grammatik ist mehrdeutig, denn das Wort

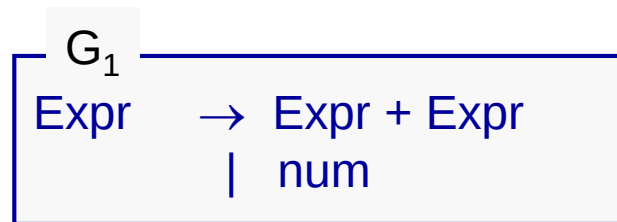
$\text{id} * \text{id} + \text{id}$

hat zwei Herleitungsbäume (äqu.: zwei Linksherleitungen)

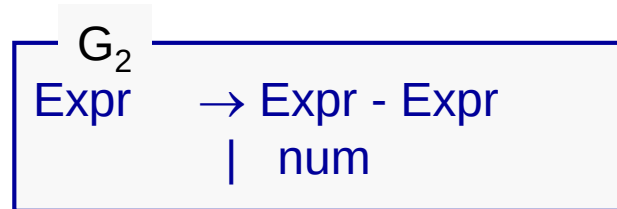


Strategien zur Mehrdeutigkeit

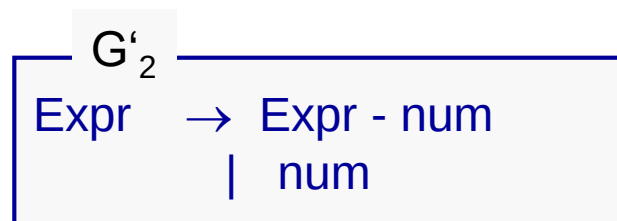
Im Sinne einer einfachen Sprachbeschreibung nimmt man Mehrdeutigkeiten einer Grammatik oft in Kauf. Beim Parsen gibt es verschiedene Strategien mit der Mehrdeutigkeit umzugehen:



Mehrdeutigkeit unerheblich,
da Addition assoziativ.



Mehrdeutigkeit wesentlich,
da Subtraktion nicht assoziativ.



Umgewandelt in
eindeutige Grammatik.

Strategien zur Mehrdeutigkeit

G_3

$\text{Expr} \rightarrow \text{Expr} + \text{Expr}$
| $\text{Expr} * \text{Expr}$
| (Expr)
| num

Mehrdeutigkeit erheblich,
z.B. $7 + 3 * 4$ hat zwei Bäume.

Neue syntaktische Kategorien
(Nonterminale) für jede
Präzedenzstufe

G'_3

$\text{Expr} \rightarrow \text{Expr} + \text{Term}$
| Term
 $\text{Term} \rightarrow \text{Term} * \text{Factor}$
| Factor
 $\text{Factor} \rightarrow (\text{Expr})$
| num

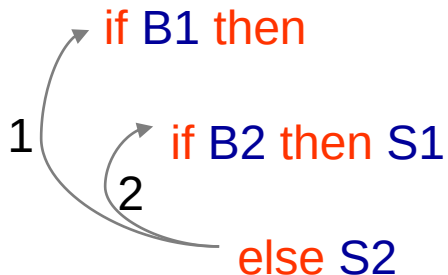
Äquivalente eindeutige Grammatik.

Strategien zur Mehrdeutigkeit

G_4

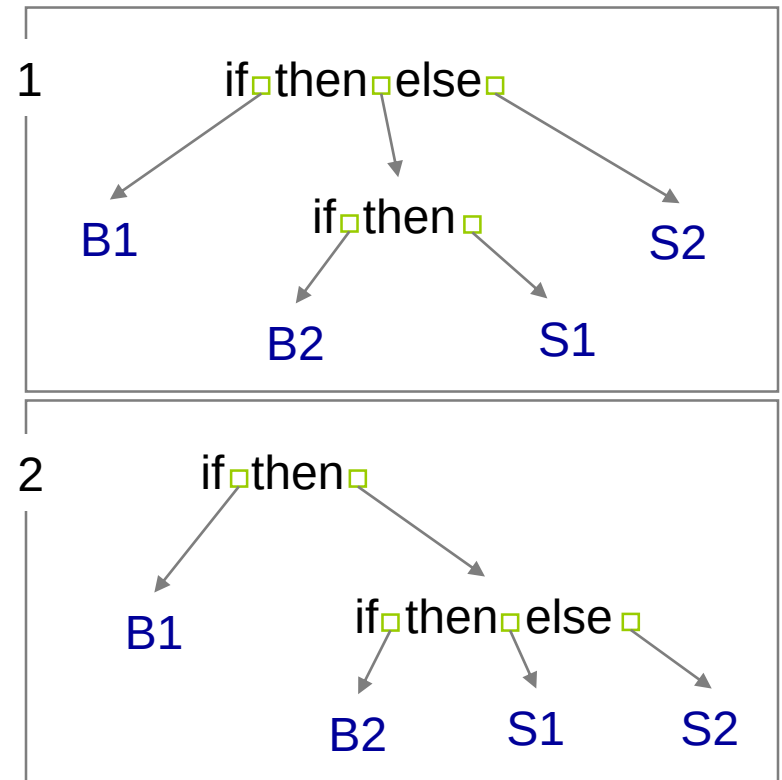
Stmt $\rightarrow$ if Bexpr then Stmt else Stmt
| if Bexpr then Stmt
| ...

Mehrdeutig:



Verabredung :

Jedes else ergänzt das
letzte unergänzte then



Chomsky-Normalform (CNF)



<http://www.chomsky.info>

Zu jeder CF Grammatik G gibt es eine äquivalente Grammatik G' , deren Regeln folgende einfache Bauart haben:

$A \rightarrow a$ // genau 1 Terminal
 $A \rightarrow BC$ // genau 2 Variablen
 $S \rightarrow \varepsilon$

a	Terminal
A, B, C	Nichtterminale
S	Startsymbol

Die Produktion $S \rightarrow \varepsilon$ ist genau dann vorhanden, wenn $\varepsilon \in L(G)$ ist.
In diesem Falle kommt S in keiner Regel rechts vor.

Der Satz von Chomsky ist ein theoretisches Ergebnis. In der Praxis ist es angenehmer, mit Regeln beliebiger Bauart spezifizieren zu dürfen.

Erreichbare Variablen

$A \in V$ heißt **erreichbar**, falls $S \Rightarrow^* \alpha A \beta$

Rekursive Charakterisierung

A **erreichbar** $\Leftrightarrow A = S$
oder $(E \rightarrow \alpha A \beta) \in P$ und E **erreichbar**.

Fast schon ein rekursives Programm. Nur für Terminierung sorgen.
Iterative Implementierung **markiert** alle erreichbaren Variablen:

Markierungs-Algorithmus

Markiere S .

DO

IF $(E \rightarrow \alpha A \beta) \in P$ und E schon markiert, **markiere** A

UNTIL in einer Runde nichts neues markiert wurde

Nicht erreichbare Variablen kann man entfernen.

Terminierende Variablen

$A \in V$ heißt **terminierend**, falls $\exists w \in T^*: A \Rightarrow^* w$

Rekursive Charakterisierung

A **terminierend** $\Leftrightarrow (A \rightarrow w) \in P$ für ein $w \in T^*$
oder $(A \rightarrow \alpha) \in P$ für ein $\alpha \in (V - \{A\} \cup T)^*$
und alle Variablen in α **terminierend**.

Markierungs-Algorithmus

Markiere alle $A \in V$ mit $(A \rightarrow w) \in P$ und $w \in T^*$.

DO

IF $(A \rightarrow \alpha) \in P$ und alle Nonterminale in α schon markiert, **markiere A**

UNTIL in einer Runde nichts neues markiert wurde

Nicht terminierende Variablen kann man entfernen.

ε -Regeln

- Transformiere Grammatik G in eine neue Grammatik G^+ mit $L(G^+) = L(G) - \{\varepsilon\}$
- Einfach alle ε -Regeln entfernen geht nicht:

- Beispiel:

– $L(G') = \{\} \neq L(G) - \{\varepsilon\}$

G

$S \rightarrow aB \mid bA \mid \varepsilon$
 $A \rightarrow aS \mid bAA$
 $B \rightarrow bS \mid aBB$

G'

$S \rightarrow aB \mid bA$
 $A \rightarrow aS \mid bAA$
 $B \rightarrow bS \mid aBB$

- Richtig:

– $L(G^+) = L(G) - \{\varepsilon\} \neq L(G)$

G⁺

$S \rightarrow aB \mid bA$
 $A \rightarrow aS \mid bAA \mid a$
 $B \rightarrow bS \mid aBB \mid b$

Kollabierende (nullable) Variablen (ε Schritt 1)

$A \in V$ heißt kollabierend, falls $A \Rightarrow^* \varepsilon$

Rekursive Charakterisierung

A kollabierend $\Leftrightarrow (A \rightarrow \varepsilon) \in P$
oder $(A \rightarrow B_1 B_2 \dots B_n) \in P$, $B_i \neq A$,
und alle B_i kollabierend).

Markierungs-Algorithmus

Markiere alle $A \in V$ mit $(A \rightarrow \varepsilon) \in P$.

DO

IF $(A \rightarrow B_1 \dots B_n) \in P$ und alle B_1 bis B_n schon markiert, markiere A

UNTIL in einer Runde nichts neues markiert wurde

Entfernung von ε -Regeln (ε Schritt 2)

Jede Regel der Form $A \rightarrow \varepsilon$ heißt ε -Regel

FOR ALL kollabierende Variable C

FOR ALL Regel der Form

$$A \rightarrow \alpha C \beta$$

füge eine neue Regel hinzu:

$$A \rightarrow \alpha \beta$$

UNTIL in einer Runde keine neue Regel hinzukam

Streiche alle ε -Regeln // nur ε -Regeln und **nicht** pauschal kollabierende Variablen

Vorsicht: Falls $A \rightarrow \alpha C \beta C \gamma$ und C kollabierend, müssen sowohl $A \rightarrow \alpha \beta C \gamma$ als auch $A \rightarrow \alpha C \beta \gamma$ und $A \rightarrow \alpha \beta \gamma$ hinzukommen.

Aus Grammatik G entsteht eine neue Grammatik G^+ ohne ε -Regeln und es gilt

$$L(G^+) = L(G) - \{\varepsilon\}$$

ε -Freiheit (ε Schritt 3)

G heißt ε -frei, falls

- keine Regel der Form $A \rightarrow \varepsilon$ vorkommt, // 1.
- **oder** ($S \rightarrow \varepsilon$ ist einzige ε -Regel, // 2.
und S kommt in keiner Produktion rechts vor).

$L(G^+) = L(G) - \{\varepsilon\}$ und G^+ **ist** ε -frei. Daher:

1. Fall: $\varepsilon \notin L(G)$. Setze $G' = G^+$.

2. Fall: $\varepsilon \in L(G)$:

G'
$S' \rightarrow \varepsilon$ $S' \rightarrow S$
alle Prod. von G^+

Mit S' als
Startsymbol von G'

Zu jeder Grammatik G gibt es eine äquivalente ε -freie Grammatik G' .

1-Variablen-Regeln

Genau **eine** Variable!

Alle Variablen-Regeln der Form $A \rightarrow B$ mit $A, B \in V$ kann man eliminieren.

Algorithmus)*

DO

FOR ALL Variablenregel $A \rightarrow B$:

FOR ALL Regel $B \rightarrow \alpha$

füge neue Regel $A \rightarrow \alpha$ hinzu

Streiche $A \rightarrow B$

UNTIL nichts mehr passiert.

)* Falls es rekursive Variablenregeln gibt
 $A \rightarrow B_1 \rightarrow B_2 \rightarrow \dots \rightarrow B_n \rightarrow A$,
wird der Algorithmus ein bisschen komplizierter

Chomsky – Normalform (CNF)

Satz: Jede kontextfreie Grammatik G lässt sich äquivalent umformen in eine Grammatik G' , deren Regeln von der Form sind:

$$\begin{aligned} A &\rightarrow a \\ A &\rightarrow BC \end{aligned}$$

Falls $\varepsilon \in L(G)$ gibt es noch die Produktion $S \rightarrow \varepsilon$ und S kommt in keiner Produktion rechts vor.

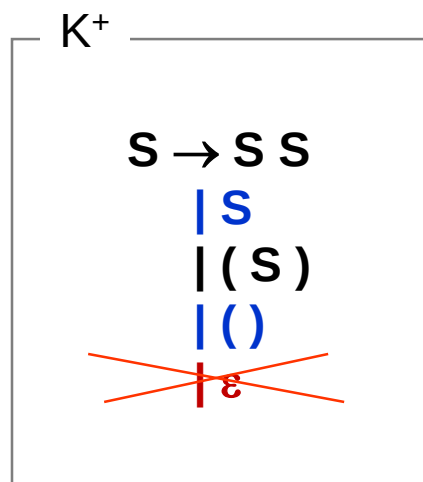
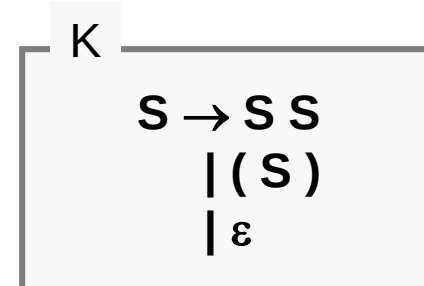
1. Entferne nicht terminierende und nicht erreichbare Variablen $)^*$
2. Mache G ε -frei und entferne alle 1-Variablen-Regeln.
3. In jeder Regel $A \rightarrow \alpha$ mit $|\alpha| > 1$ ersetze Terminalsymbol a durch neue Variable X_a und füge die Regel $X_a \rightarrow a$ hinzu.
4. Ersetze jede Regel der Form $A \rightarrow B_1 B_2 \dots B_n$ mit Hilfe neuer Variablen $C_1 \dots C_{n-2}$ durch

$$A \rightarrow B_1 C_1, \quad C_1 \rightarrow B_2 C_2 \quad \dots \quad C_{n-2} \rightarrow B_{n-1} B_n$$

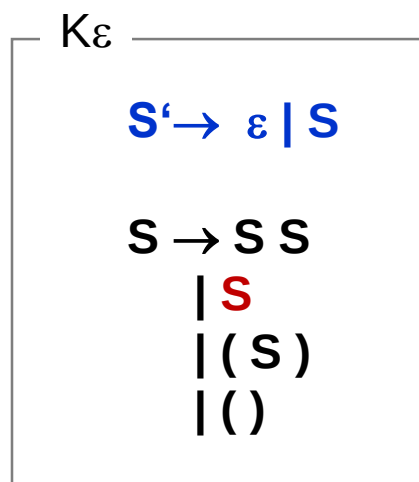
$)^*$ optionaler Schritt. Kann nach 2. ggf. wiederholt werden

Beispiel: Gewinnung einer CNF – Anfang

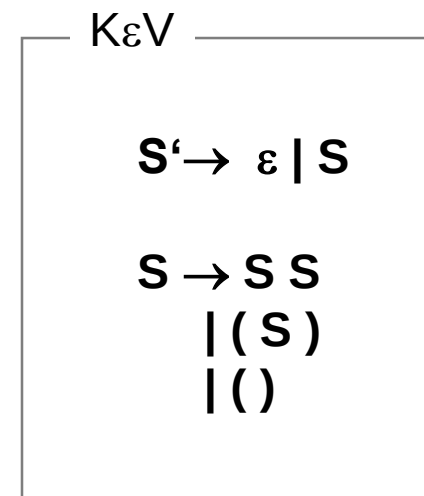
- Wir wandeln die Kammersprache K in CNF um
- Jede Variable ist erreichbar und terminierend
 - Das ist normal für realistische Grammatiken! (1)
- S ist kollabierend. Wir erhalten nacheinander (2)
 - K^+ durch Entfernen der ε -Regeln
 - K_ε als zu K äquivalente ε -freie Grammatik
 - $K_{\varepsilon V}$ durch Elimination von Variablenregeln



$$L(K^+) \neq L(K)$$



$$L(K_\varepsilon) = L(K)$$



$$L(K_{\varepsilon V}) = L(K)$$

Beispiel: Gewinnung einer CNF – Schluss

▪ Wir wandeln die Kammersprache K in CNF um

- Wir haben bereits $K_{\varepsilon V}$ zu K äquivalent
 - alle Variablen terminierend und ε -frei
 - ohne 1-Variablenregeln (fast)
- Wir ersetzen nun Terminale durch Variablen (3)
- und verkürzen rechte Seiten (4)

$$\begin{array}{l}
 K_{\varepsilon V} \\
 S' \rightarrow \varepsilon \mid S \\
 S \rightarrow S S \\
 \quad \mid (S) \\
 \quad \mid ()
 \end{array}$$

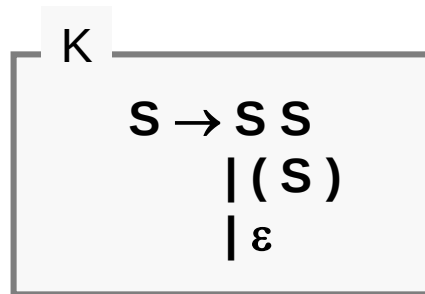
$$\begin{array}{l}
 K_{\varepsilon VT} \\
 S' \rightarrow \varepsilon \mid S \\
 S \rightarrow S S \\
 \quad \mid A S Z \\
 \quad \mid A Z \\
 A \rightarrow (\\
 Z \rightarrow)
 \end{array}$$

$$\begin{array}{l}
 K \\
 S \rightarrow S S \\
 \quad \mid (S) \\
 \quad \mid \varepsilon
 \end{array}$$

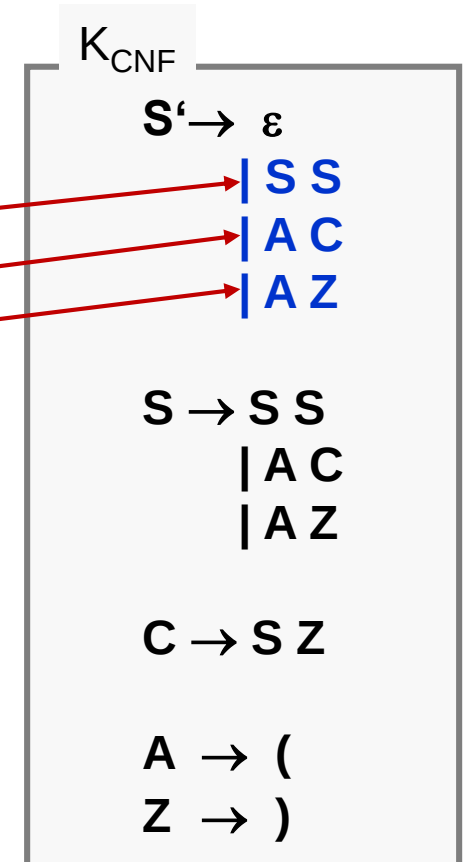
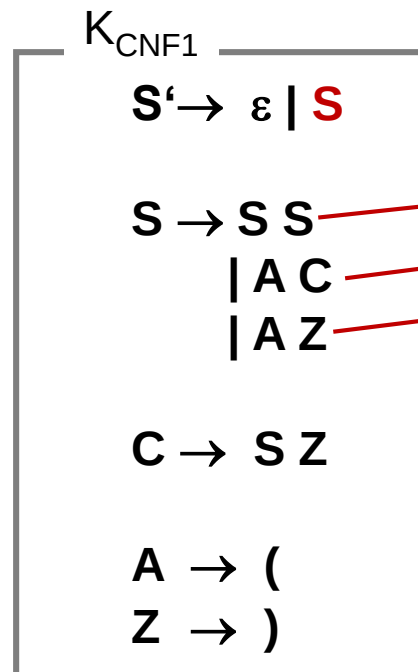
$$\begin{array}{l}
 K_{CNF1} \\
 S' \rightarrow \varepsilon \mid S \\
 S \rightarrow S S \\
 \quad \mid A C \\
 \quad \mid A Z \\
 C \rightarrow S Z \\
 A \rightarrow (\\
 Z \rightarrow)
 \end{array}$$

Die Normalform – jetzt wirklich Schluss

- Eine 1-Variablenregel ist noch zu entfernen:



Ausgangs
grammatik



Wortproblem für kontextfreie Sprachen

Sei G eine CF Grammatik, $A \in V$ und $w = a_1 a_2 \dots a_n \in T^*$.

Gilt $A \Rightarrow^* w$ also $w \in L(G)$ für $A = S$?

Wir haben einen Algorithmus, um G in Chomsky-Normalform G_{CNF} zu überführen. Dann wenden wir den folgenden rekursiven Algorithmus an:

CYK-Algorithmus - rekursiv

$A \Rightarrow^* w \iff$ $w = \varepsilon$, $A = S$ und $S \rightarrow \varepsilon \in G_{\text{CNF}}$
oder $w = a \in T$ und $A \rightarrow a \in G_{\text{CNF}}$
oder $|w| > 1$ und es gibt ein $1 \leq k < n$ mit
 $A \rightarrow BC \in G_{\text{CNF}}$ und $B \Rightarrow^* a_1 \dots a_k$
und $C \Rightarrow^* a_{k+1} \dots a_n$

Jede kontextfreie Sprache ist entscheidbar.

Der Algorithmus ist so noch ziemlich ineffizient, da viele Teilprobleme mehrfach berechnet werden.

CYK-Algorithmus - imperativ

Imperativ implementiert man den Algorithmus durch **dynamisches Programmieren**.

Gegeben $w = a_1 \dots a_n \in T^*$. Für jedes $1 \leq i, k \leq n$ sei

$$M_{ik} = \{ A \in V \mid A \Rightarrow^* a_i \dots a_{i+k-1} \}$$

Dann gilt:

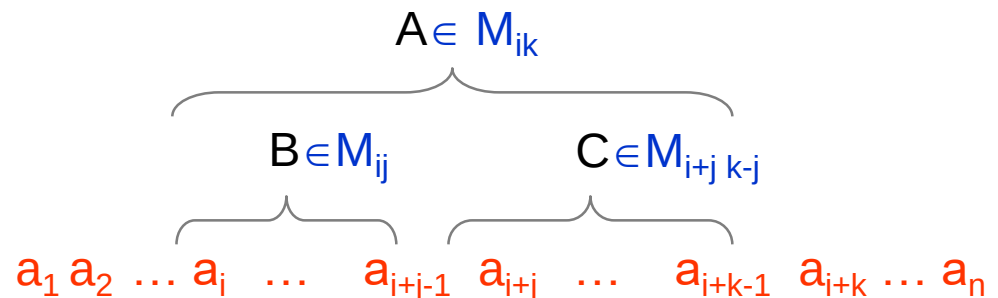
$$w \in L(G) \Leftrightarrow S \in M_{1n}$$

Ist G in Chomsky-Normalform, so kann man die M_{ik} in eine Tabelle eintragen.

5	S?				
4	M_{14}	M_{24}			
3	M_{13}	M_{23}	M_{33}		
2	M_{12}	M_{22}	M_{32}	M_{42}	
1	M_{11}	M_{21}	M_{31}	M_{41}	M_{51}
M_{ik}	a_1	a_2	a_3	a_4	a_5

$$M_{i1} = \{ A \in V \mid A \rightarrow a_i \in G \}$$

$$M_{ik} = \{ A \in V \mid \exists_{1 \leq j < k} \exists A \rightarrow BC \in G: B \in M_{ij}, C \in M_{i+j, k-j} \}$$



CYK- Algorithmus – ein Beispiel

- Gegeben sei die Sprache der nichtleeren Klammerausdrücke
- Gefragt: Ist $((())()) \in L(G)$?

8									
7									
6									
5									
4									
3									
2									
1									
M_{ij}	(	(	(	)	)	(	)	)	

K_{CNF}^+
$S \rightarrow S S$
$\quad A C$
$\quad A Z$
$C \rightarrow S Z$
$A \rightarrow ($
$Z \rightarrow)$

CYK- Algorithmus – ein Beispiel

- Gegeben sei die Sprache der nichtleeren Klammerausdrücke
- Gefragt: Ist $((())()) \in L(G)$?

8								
7								
6								
5								
4								
3								
2								
1	A	A	A	Z	Z	A	Z	Z
M_{ij}	(	(	(	)	)	(	)	)

K_{CNF}^+
$S \rightarrow SS$
$\quad AC$
$\quad AZ$
$C \rightarrow SZ$
$A \rightarrow ($
$Z \rightarrow)$

CYK- Algorithmus – ein Beispiel

- Gegeben sei die Sprache der nichtleeren Klammerausdrücke
- Gefragt: Ist $((())()) \in L(G)$?

8								
7								
6								
5								
4								
3								
2	-	-	S	-	-	S	-	
1	A	A	A	Z	Z	A	Z	Z
M_{ij}	(	(	(	)	)	(	)	)

K_{CNF}^+
$S \rightarrow SS$
$ AC$
$ AZ$
$C \rightarrow SZ$
$A \rightarrow ($
$Z \rightarrow)$

CYK- Algorithmus – ein Beispiel

- Gegeben sei die Sprache der nichtleeren Klammerausdrücke
- Gefragt: Ist $((())()) \in L(G)$?

8								
7								
6								
5								
4								
3	-	-	C	-	-	C		
2	-	-	S	-	-	S	-	
1	A	A	A	Z	Z	A	Z	Z
M_{ij}	(	(	(	)	)	(	)	)

K_{CNF}^+
$S \rightarrow S S$
$\quad A C$
$\quad A Z$
$C \rightarrow S Z$
$A \rightarrow ($
$Z \rightarrow)$

CYK- Algorithmus – ein Beispiel

- Gegeben sei die Sprache der nichtleeren Klammerausdrücke
- Gefragt: Ist $((())()) \in L(G)$?

8								
7								
6								
5								
4	-	S	-	-	-			
3	-	-	C	-	-	C		
2	-	-	S	-	-	S	-	
1	A	A	A	Z	Z	A	Z	Z
M_{ij}	(	(	(	)	)	(	)	)

K_{CNF}^+
$S \rightarrow SS$
$\quad AC$
$\quad AZ$
$C \rightarrow SZ$
$A \rightarrow ($
$Z \rightarrow)$

CYK- Algorithmus – ein Beispiel

- Gegeben sei die Sprache der nichtleeren Klammerausdrücke
- Gefragt: Ist $((())()) \in L(G)$?

8								
7								
6								
5	-	-	-	-				
4	-	S	-	-	-			
3	-	-	C	-	-	C		
2	-	-	S	-	-	S	-	
1	A	A	A	Z	Z	A	Z	Z
M_{ij}	(	(	(	)	)	(	)	)

K_{CNF}^+
$S \rightarrow S S$
$\quad A C$
$\quad A Z$
$C \rightarrow S Z$
$A \rightarrow ($
$Z \rightarrow)$

CYK- Algorithmus – ein Beispiel

- Gegeben sei die Sprache der nichtleeren Klammerausdrücke
- Gefragt: Ist $((())()) \in L(G)$?

8								
7								
6	-	S	-					
5	-	-	-	-				
4	-	S	-	-	-			
3	-	-	C	-	-	C		
2	-	-	S	-	-	S	-	
1	A	A	A	Z	Z	A	Z	Z
M_{ij}	(	(	(	)	)	(	)	)

K_{CNF}^+
$S \rightarrow SS$
$\quad AC$
$\quad AZ$
$C \rightarrow SZ$
$A \rightarrow ($
$Z \rightarrow)$

CYK- Algorithmus – ein Beispiel

- Gegeben sei die Sprache der nichtleeren Klammerausdrücke
- Gefragt: Ist $((())()) \in L(G)$?

8								
7	-	C						
6	-	S	-					
5	-	-	-	-				
4	-	S	-	-	-			
3	-	-	C	-	-	C		
2	-	-	S	-	-	S	-	
1	A	A	A	Z	Z	A	Z	Z
M_{ij}	(	(	(	)	)	(	)	)

K_{CNF}^+
$S \rightarrow S S$
$\quad A C$
$\quad A Z$
$C \rightarrow S Z$
$A \rightarrow ($
$Z \rightarrow)$

CYK- Algorithmus – ein Beispiel

- Gegeben sei die Sprache der nichtleeren Klammerausdrücke
- Gefragt: Ist $((())()) \in L(G)$?

$S \in M_{18} \iff W \in L(K^+_{\text{CNF}})$

8	S								
7	-	C							
6	-	S	-						
5	-	-	-	-					
4	-	S	-	-	-				
3	-	-	C	-	-	C			
2	-	-	S	-	-	S	-		
1	A	A	A	Z	Z	A	Z	Z	
M_{ij}	(	(	(	)	)	(	)	)	

K^+_{CNF}

$S \rightarrow S S$ $\quad A C$ $\quad A Z$
$C \rightarrow S Z$
$A \rightarrow ($ $Z \rightarrow)$

CYK- Algorithmus – ein Beispiel - Ableitung

- Gegeben sei die Sprache der nichtleeren Klammerausdrücke
- Wie entsteht der Ableitungsbaum?

$S \in M_{18} \iff w \in L(K^+_{\text{CNF}})$

8	S								
7	-	C							
6	-	S	-						
5	-	-	-	-					
4	-	S	-	-	-				
3	-	-	C	-	-	C			
2	-	-	S	-	-	S	-		
1	A	A	A	Z	Z	A	Z	Z	
M_{ij}	(	(	(	)	)	(	)	)	

K^+_{CNF}

$S \rightarrow S S$
A C
A Z
$C \rightarrow S Z$
$A \rightarrow ($
$Z \rightarrow)$

CYK- Algorithmus – ein Beispiel - Ableitung

- Gegeben sei die Sprache der nichtleeren Klammerausdrücke
- Wie entsteht der Ableitungsbaum?

$S \in M_{18} \iff w \in L(K^+_{\text{CNF}})$

8	S								
7	-	C							
6	-	S	-						
5	-	-	-	-					
4	-	S	-	-	-				
3	-	-	C	-	-	C			
2	-	-	S	-	-	S	-		
1	A	A	A	Z	Z	A	Z	Z	
M_{ij}	(	(	(	)	)	(	)	)	

Note: Red arrows indicate a path from S at (8,1) to C at (7,2) to S at (6,2) to A at (1,1).

K^+_{CNF}

$S \rightarrow S S$ $\quad A C$ $\quad A Z$
$C \rightarrow S Z$
$A \rightarrow ($ $Z \rightarrow)$

CYK- Algorithmus – ein Beispiel - Ableitung

- Gegeben sei die Sprache der nichtleeren Klammerausdrücke
- Wie entsteht der Ableitungsbaum?

$S \in M_{18} \iff w \in L(K_{\text{CNF}}^+)$

8	S							
7	-	C						
6	-	S	-					
5	-	-	-	-				
4	-	S	-	-	-			
3	-	-	C	-	-	C		
2	-	-	S	-	-	S	-	
1	A	A	A	Z	Z	A	Z	Z
M_{ij}	(	(	(	)	)	(	)	)

Red arrows indicate the derivation path: (8,1) S → (7,2) C → (6,2) S → (4,3) S → (2,7) S → (1,1) A → (1,9) Z.

K_{CNF}^+
$S \rightarrow S S$ $\quad A C$ $\quad A Z$
$C \rightarrow S Z$
$A \rightarrow ($ $Z \rightarrow)$

CYK- Algorithmus – ein Beispiel - Ableitung

- Gegeben sei die Sprache der nichtleeren Klammerausdrücke
- Wie entsteht der Ableitungsbaum?

$S \in M_{18} \iff w \in L(K_{CNF}^+)$

8	S							
7	-	C						
6	-	S	-					
5	-	-	-	-				
4	-	S	-	-	-			
3	-	-	C	-	-	C		
2	-	-	S	-	-	S	-	
1	A	A	A	Z	Z	A	Z	Z
M_{ij}	(	(	(	)	)	(	)	)

Red arrows indicate the derivation path: (8,1) S → (7,2) C → (6,3) S → (4,2) S → (3,7) C → (2,6) S → (1,1) A → (1,9) Z.

K_{CNF}^+
$S \rightarrow S S$ $\quad A C$ $\quad A Z$
$C \rightarrow S Z$
$A \rightarrow ($
$Z \rightarrow)$

CYK- Algorithmus – ein Beispiel - Ableitung

- Gegeben sei die Sprache der nichtleeren Klammerausdrücke
- Wie entsteht der Ableitungsbaum?

$S \in M_{18} \iff W \in L(K_{\text{CNF}}^+)$

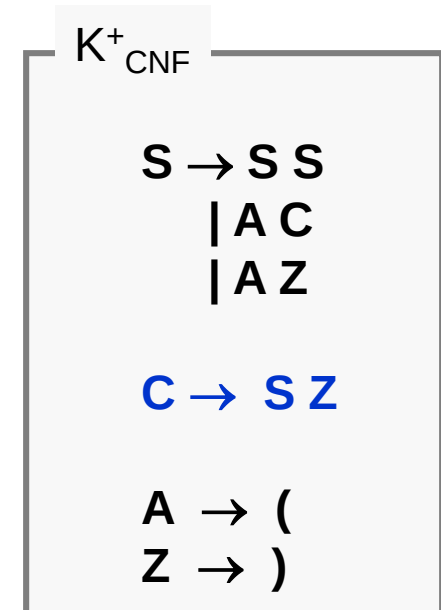
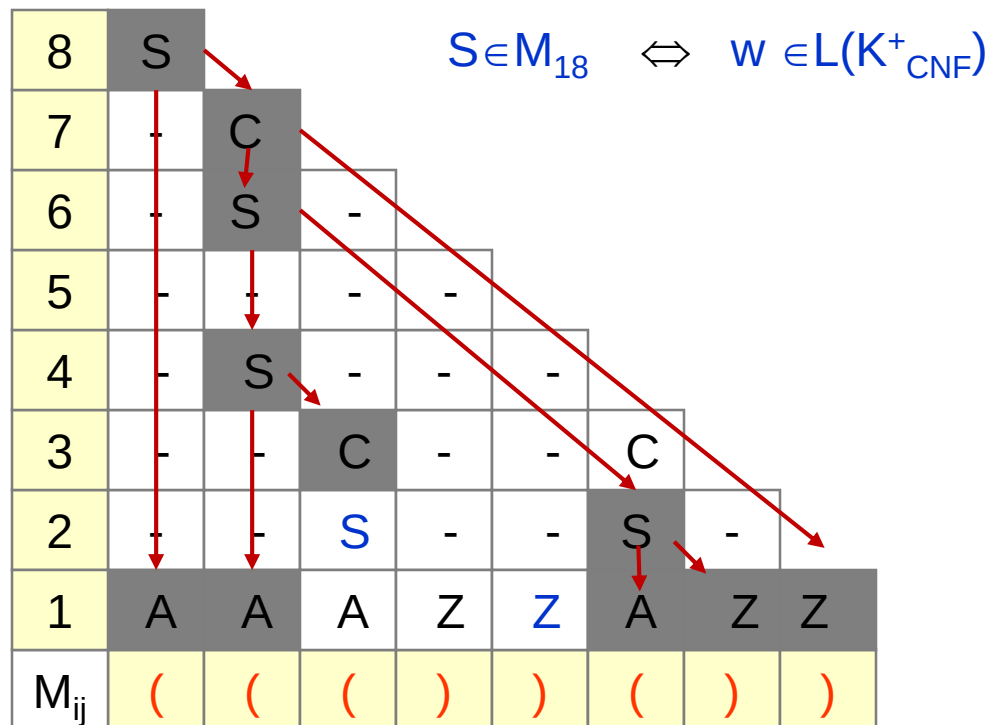
8	S								
7	-	C							
6	-	S	-						
5	-	-	-	-					
4	-	S	-	-	-				
3	-	-	C	-	-	C			
2	-	-	S	-	-	S	-		
1	A	A	A	Z	Z	A	Z	Z	
M_{ij}	(	(	(	)	)	(	)	)	

K_{CNF}^+

$S \rightarrow SS$
AC
AZ
$C \rightarrow SZ$
$A \rightarrow ($
$Z \rightarrow)$

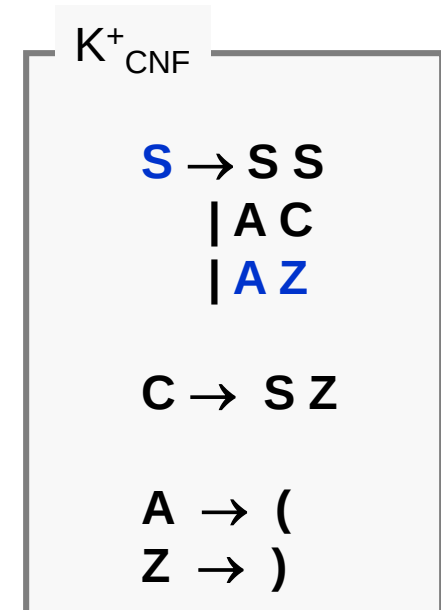
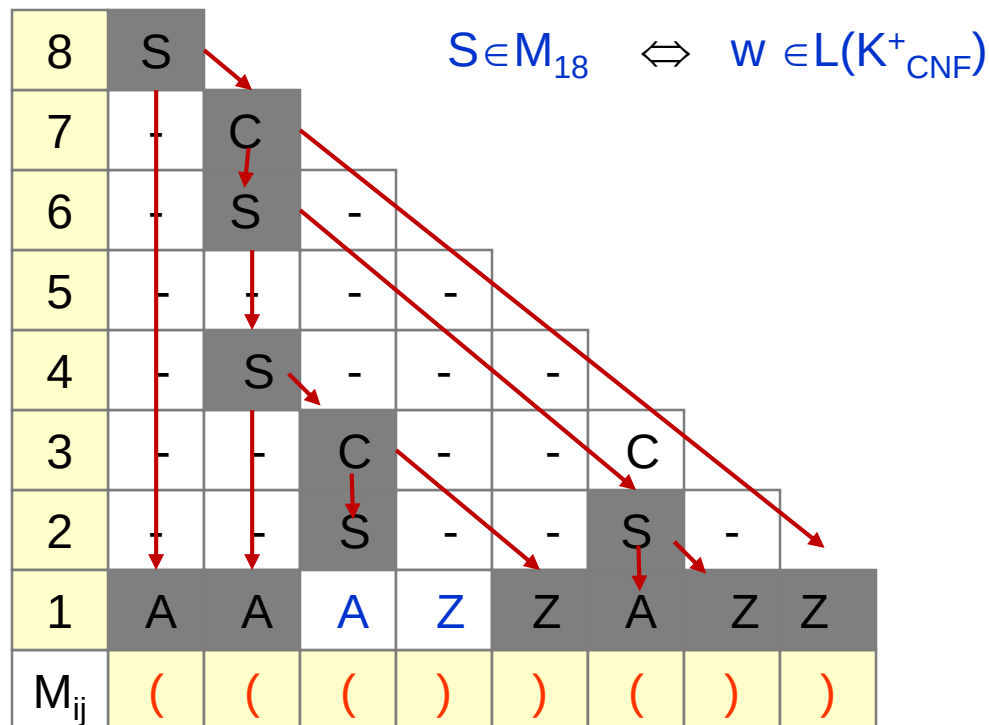
CYK- Algorithmus – ein Beispiel - Ableitung

- Gegeben sei die Sprache der nichtleeren Klammerausdrücke
- Wie entsteht der Ableitungsbaum?



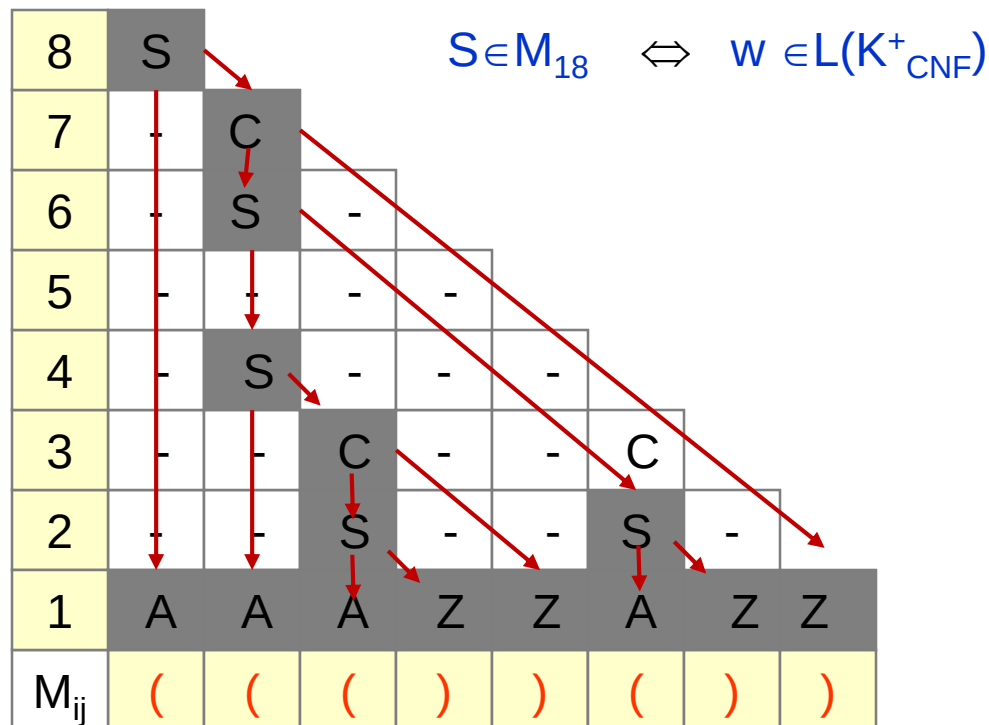
CYK- Algorithmus – ein Beispiel - Ableitung

- Gegeben sei die Sprache der nichtleeren Klammerausdrücke
- Wie entsteht der Ableitungsbaum?



CYK- Algorithmus – ein Beispiel - Ableitung

- Gegeben sei die Sprache der nichtleeren Klammerausdrücke
- So entsteht der Ableitungsbaum!



$$K_{\text{CNF}}^+$$

$S \rightarrow S S$
$\quad A C$
$\quad A Z$
$C \rightarrow S Z$
$A \rightarrow ($
$Z \rightarrow)$

Grenzen von CF-Sprachen

- Gibt es Sprachen, die nicht kontextfrei sind?
 - ja, z.B. $\{a^n b^n c^n \mid n \geq 0\}$
- Wie kann man das beweisen?
 - Pumping Lemma für CF Sprachen – kommt gleich
- Sind CF-Sprachen unter $\cap$ abgeschlossen?
 - **Nein**, denn

$$\underbrace{\{a^n b^n c^n \mid n \geq 0\}}_{\text{Nicht CF!}} = \underbrace{\{a^n b^n c^k \mid n \geq 0, k \geq 0\}}_{\text{CF}} \cap \underbrace{\{a^k b^n c^n \mid n \geq 0, k \geq 0\}}_{\text{CF}}$$

- Unter Komplement?
 - **Nein**, denn sie sind unter $\cup$ abgeschlossen, und mit Komplement und Vereinigung kann man den Schnitt ausdrücken:

$$L \cap M = \Sigma^* - ((\Sigma^* - L) \cup (\Sigma^* - M))$$

Pumping Lemma für CF-Sprachen

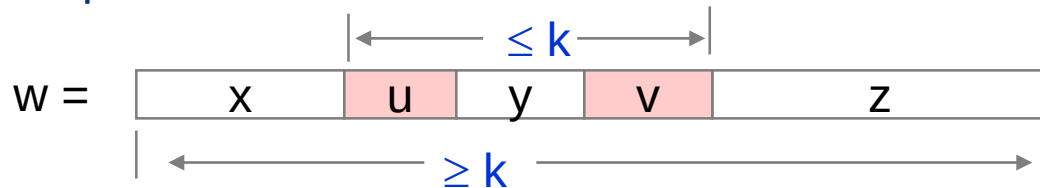
Für jede CF-Sprache L gibt es eine Zahl k , so daß jedes Wort $w \in L$ mit $|w| \geq k$ sich zerlegen läßt als

$$w = x u y v z$$

so dass

- $0 < |uv| \leq |uyv| \leq k$
- $\forall n \in \mathbb{N}: x u^n y v^n z \in L$

Also jedes große ($\geq k$) Wort hat ein kleines ($\leq k$) Teilwort uyv , das sich ringförmig aufpumpen lässt:



x	y	z
---	---	---

 $\in L$ // abpumpen

x	u	u	y	v	v	z
---	---	---	---	---	---	---

 $\in L$ // aufpumpen

x	u	u	u	y	v	v	v	z
---	---	---	---	---	---	---	---	---

 $\in L$ // mehr

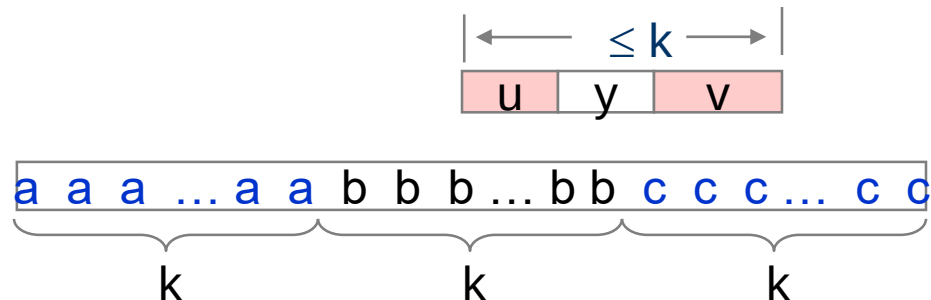
Anwendung des CF-Pumping Lemma

$L = \{a^n b^n c^n \mid n \geq 0\}$ ist nicht CF

- **Angenommen** L sei CF, dann gäbe es ein k wie im PL.
- Betrachte $w = a^k b^k c^k \in L$.

- Gemäß PL lässt sich dies zerlegen:

$$a^k b^k c^k = x u y v z \text{ mit } |u y v| \leq k$$



- In uyv kommt entweder kein a oder kein c vor. OBdA kein a .
- Wegen $|uv| > 0$ hat dann xyz zu viele a 's um in L zu sein.

➤ **Widerspruch** zum PL.

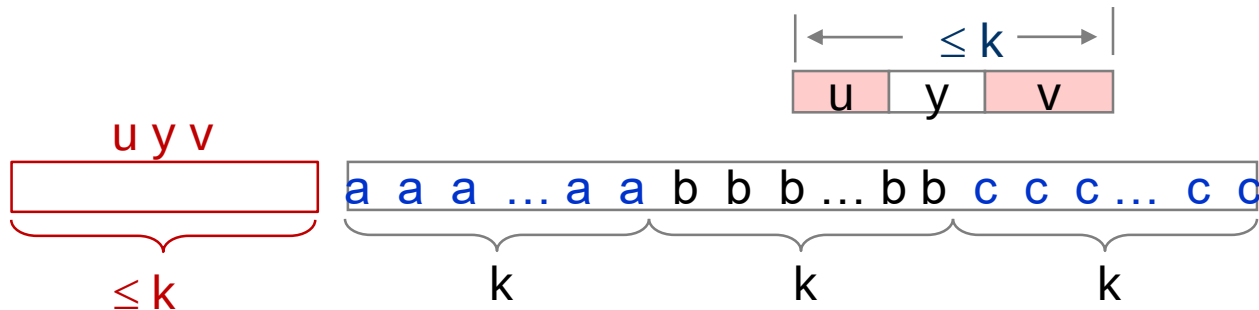
Anwendung des CF-Pumping Lemma

$L = \{a^n b^n c^n \mid n \geq 0\}$ ist nicht CF

- **Angenommen** L sei CF, dann gäbe es ein k wie im PL.
- Betrachte $w = a^k b^k c^k \in L$.

- Gemäß PL lässt sich dies zerlegen:

$$a^k b^k c^k = x u y v z \text{ mit } |u y v| \leq k$$



- In uyv kommt entweder kein a oder kein c vor. OBdA kein a .
- Wegen $|uv| > 0$ hat dann xyz zu viele a 's um in L zu sein.

➤ **Widerspruch** zum PL.

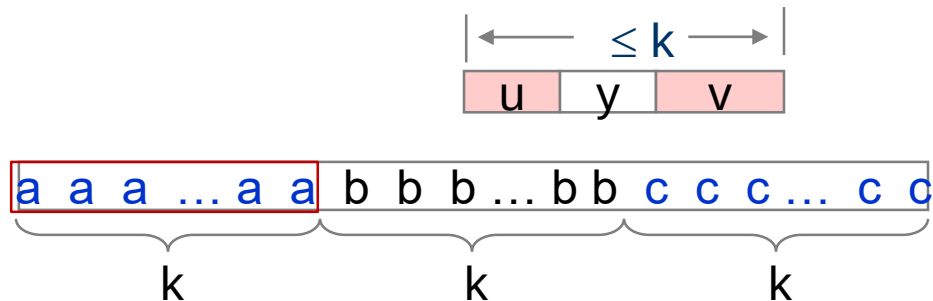
Anwendung des CF-Pumping Lemma

$L = \{a^n b^n c^n \mid n \geq 0\}$ ist nicht CF

- **Angenommen** L sei CF, dann gäbe es ein k wie im PL.
- Betrachte $w = a^k b^k c^k \in L$.

- Gemäß PL lässt sich dies zerlegen:

$$a^k b^k c^k = x u y v z \text{ mit } |u y v| \leq k$$



- In uyv kommt entweder kein a oder kein c vor. OBdA kein a .
- Wegen $|uv| > 0$ hat dann xyz zu viele a 's um in L zu sein.

➤ **Widerspruch** zum PL.

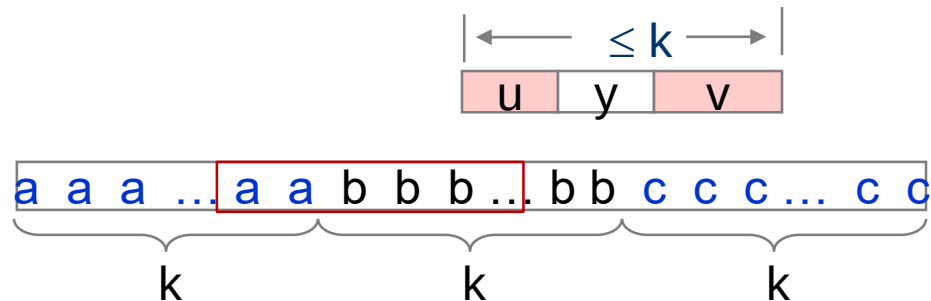
Anwendung des CF-Pumping Lemma

$L = \{a^n b^n c^n \mid n \geq 0\}$ ist nicht CF

- **Angenommen** L sei CF, dann gäbe es ein k wie im PL.
- Betrachte $w = a^k b^k c^k \in L$.

- Gemäß PL lässt sich dies zerlegen:

$$a^k b^k c^k = x u y v z \text{ mit } |u y v| \leq k$$



- In uyv kommt entweder kein a oder kein c vor. OBdA kein a .
- Wegen $|uv| > 0$ hat dann xyz zu viele a 's um in L zu sein.

➤ **Widerspruch** zum PL.

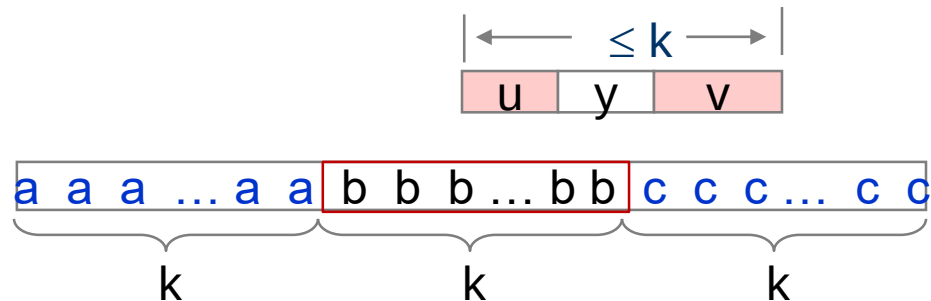
Anwendung des CF-Pumping Lemma

$L = \{a^n b^n c^n \mid n \geq 0\}$ ist nicht CF

- **Angenommen** L sei CF, dann gäbe es ein k wie im PL.
- Betrachte $w = a^k b^k c^k \in L$.

- Gemäß PL lässt sich dies zerlegen:

$$a^k b^k c^k = x u y v z \text{ mit } |u y v| \leq k$$



- In uyv kommt entweder kein a oder kein c vor. OBdA kein a .
- Wegen $|uv| > 0$ hat dann xyz zu viele a 's um in L zu sein.

➤ **Widerspruch** zum PL.

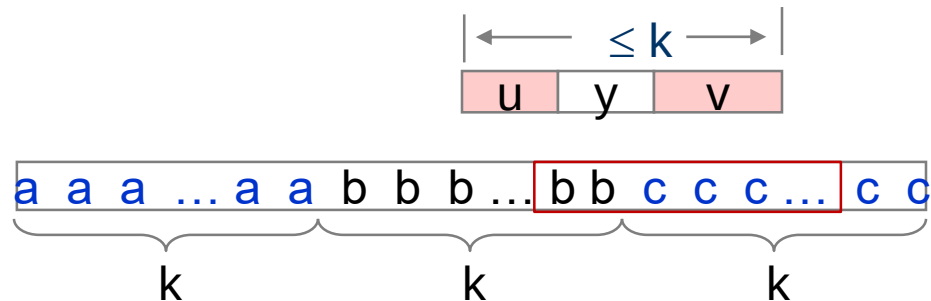
Anwendung des CF-Pumping Lemma

$L = \{a^n b^n c^n \mid n \geq 0\}$ ist nicht CF

- **Angenommen** L sei CF, dann gäbe es ein k wie im PL.
- Betrachte $w = a^k b^k c^k \in L$.

- Gemäß PL lässt sich dies zerlegen:

$$a^k b^k c^k = x u y v z \text{ mit } |u y v| \leq k$$

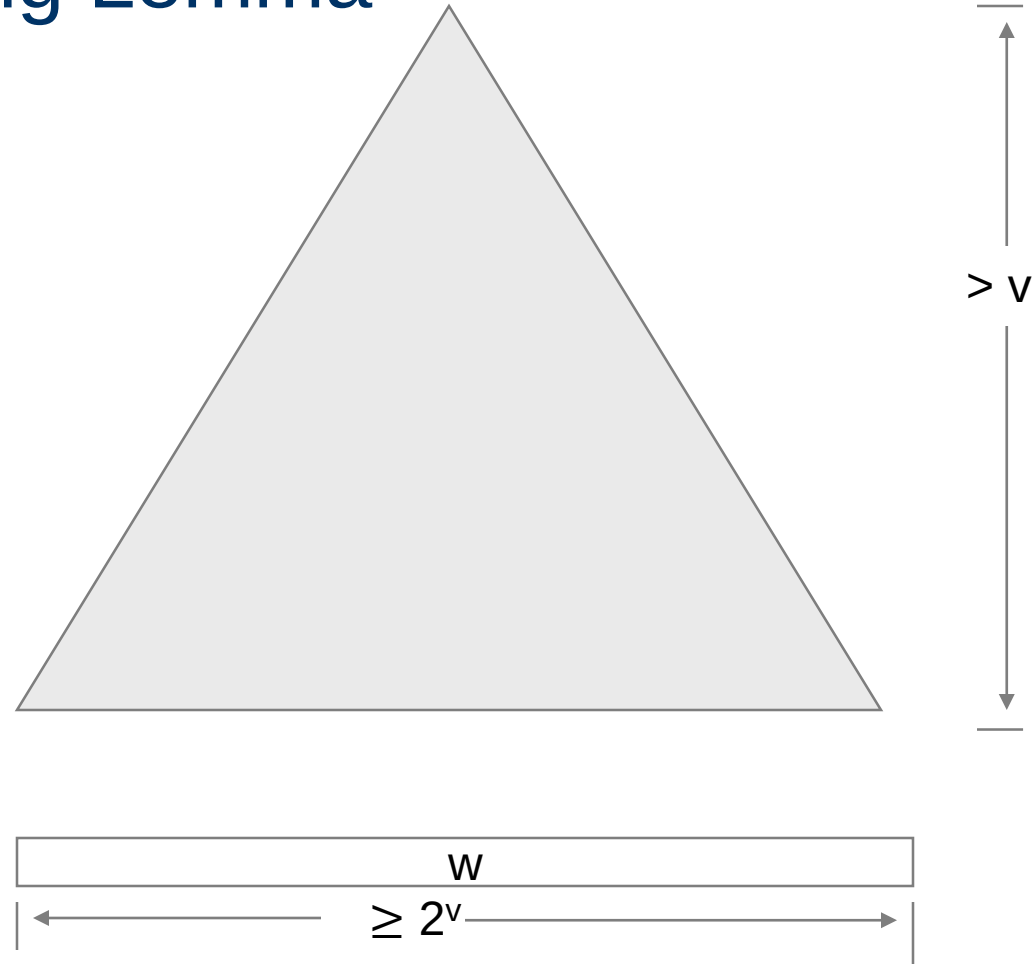


- In uyv kommt entweder kein a oder kein c vor.
- Wegen $|uv| > 0$ hat dann xyz zu viele a 's (oder b 's oder c 's) um in L zu sein.

➤ **Widerspruch** zum PL.

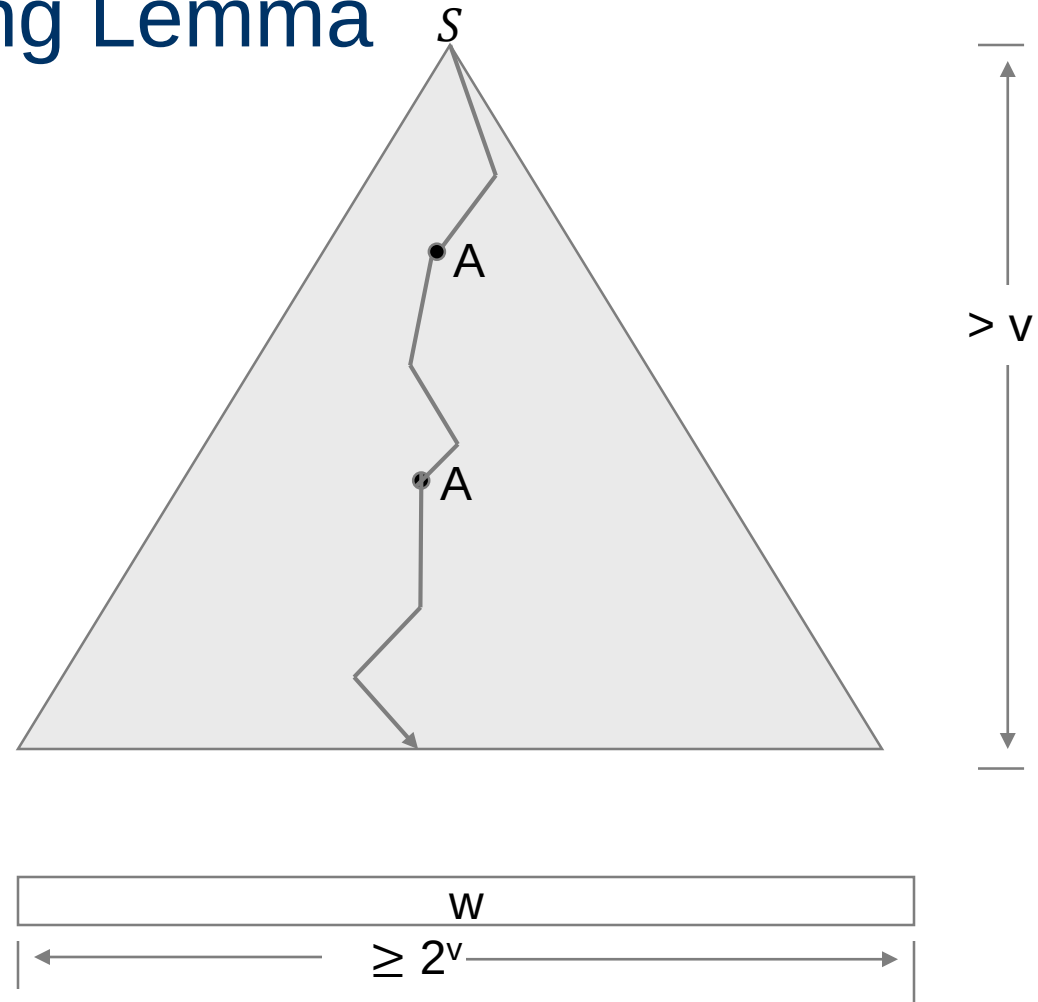
Beweis des Pumping Lemma

- Sei G eine Grammatik in **Chomsky-Normalform** für L mit $v-1$ Variablen.
- Setze $k=2^v$.
- Ist $w \in L(G)$ mit $|w| \geq k$, dann hat der Herleitungsbaum für w Tiefe $> v$.



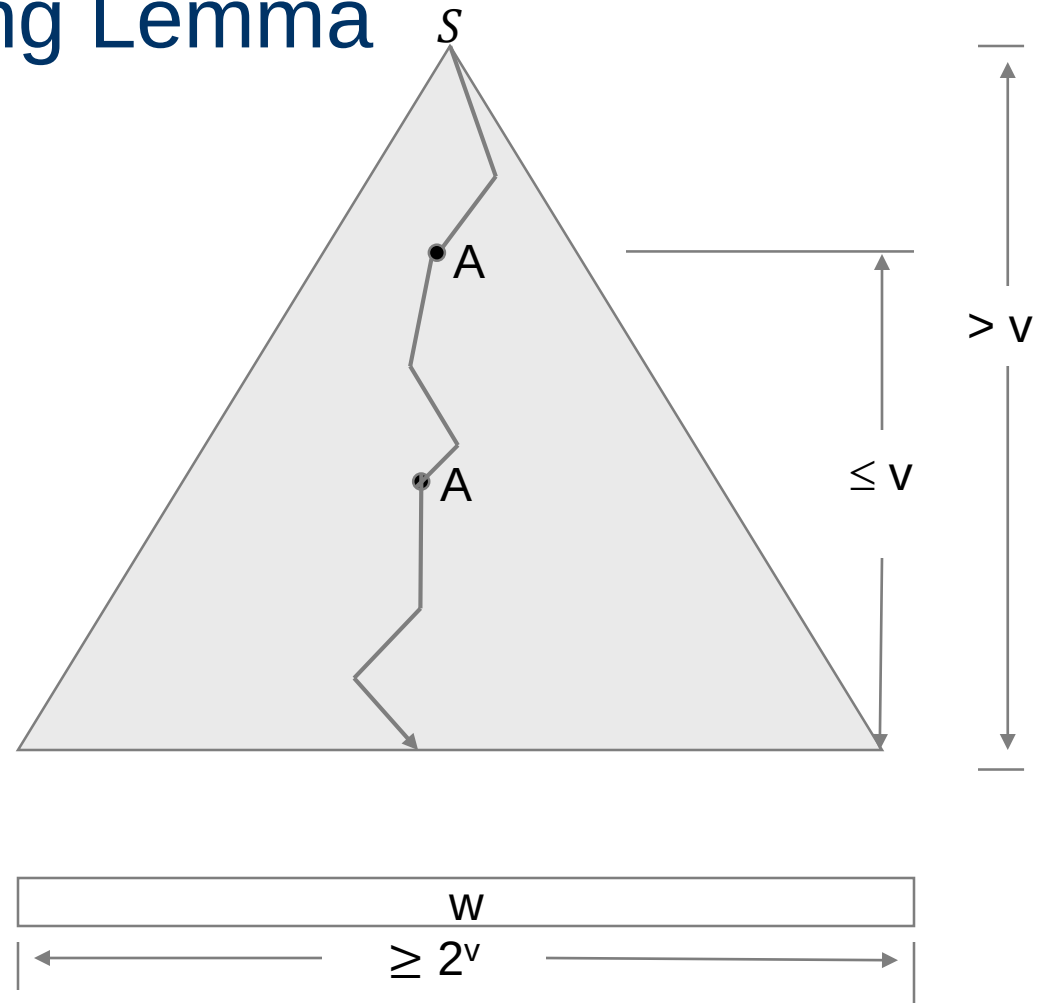
Beweis des Pumping Lemma

- Sei G eine Grammatik in **Chomsky-Normalform** für L mit $v-1$ Variablen.
- Setze $k=2^v$.
- Ist $w \in L(G)$ mit $|w| \geq k$, dann hat der Herleitungsbaum für w Tiefe $> v$.
- Es gibt also einen Pfad zur Wurzel, auf dem ein Nonterminal A zweimal vorkommt.



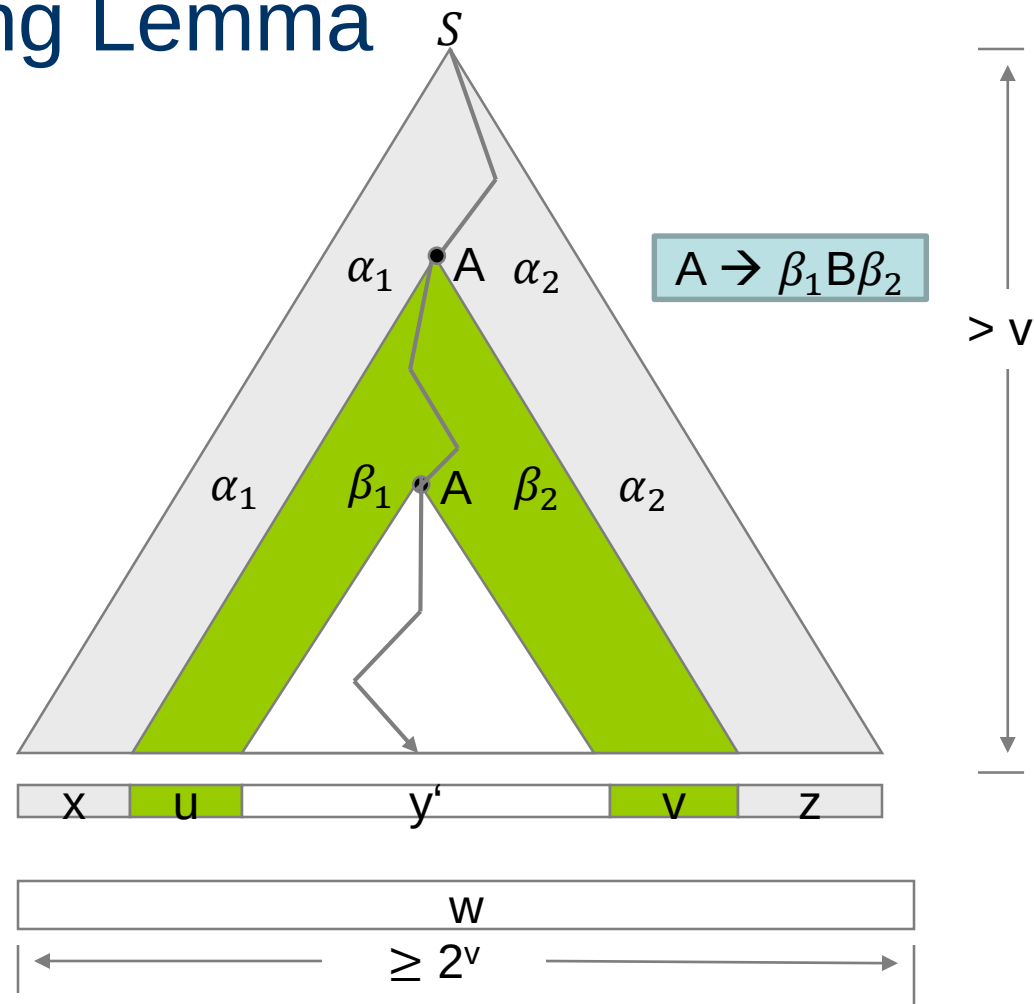
Beweis des Pumping Lemma

- Sei G eine Grammatik in **Chomsky-Normalform** für L mit $v-1$ Variablen.
- Setze $k=2^v$.
- Ist $w \in L(G)$ mit $|w| \geq k$, dann hat der Herleitungsbaum für w Tiefe $> v$.
- Es gibt also einen Pfad zur Wurzel, auf dem ein Nonterminal A zweimal vorkommt. Man kann die beiden A sogar von Tiefe $\leq v$ wählen.



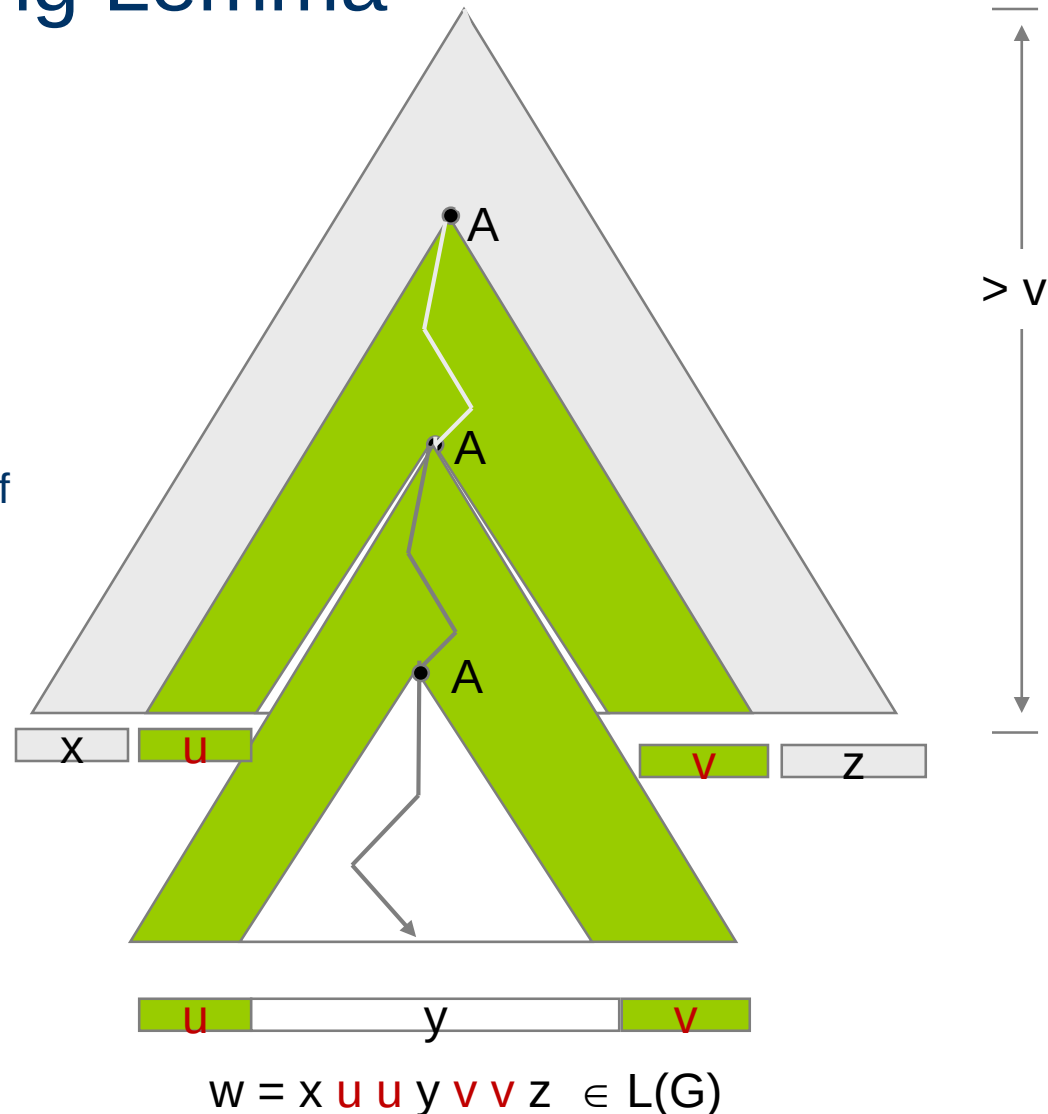
Beweis des Pumping Lemma

- Sei G eine Grammatik in **Chomsky-Normalform** für L mit $v-1$ Variablen.
- Setze $k=2^v$.
- Ist $w \in L(G)$ mit $|w| \geq k$, dann hat der Herleitungsbaum für w Tiefe $> v$.
- Es gibt also einen Pfad zur Wurzel, auf dem ein Nonterminal A zweimal vorkommt. Man kann die beiden A sogar von Tiefe $\leq v$ wählen.
- w zerlegt sich also als $w = x u y v z$ wie in der Figur



Beweis des Pumping Lemma

- Sei G eine Grammatik in **Chomsky-Normalform** für L mit $v-1$ Variablen.
- Setze $k=2^v$.
- Ist $w \in L(G)$ mit $|w| \geq k$, dann hat der Herleitungsbaum für w Tiefe $> v$.
- Es gibt also einen Pfad zur Wurzel, auf dem ein Nonterminal A zweimal vorkommt. Man kann die beiden A sogar von Tiefe $\leq v$ wählen.
- w zerlegt sich also als $w = x u y v z$ wie in der Figur
- Wir könnten den Baum unter dem ersten Vorkommen von A auch nochmal verwenden
- und nochmal ... $x u u u y v v v z \in L$



- 





Abschluss unter Operationen



- Sind L_1 , L_2 , L kontextfrei, dann auch
 - $L_1 + L_2$ (Vereinigung)
 - $L_1 L_2$ (Verkettung)
 - L^* (Kleene-Stern)
- aber: kontextfreie Sprachen sind **nicht** abgeschlossen unter
 - $\Sigma^* - L$ (Komplement)
 - $L_1 \cap L_2$ (Schnitt)
- Gegenbeispiel :
 - $L_1 = \{ a^k b^m c^n \mid k=m \}$ **Kontextfrei**
 - $L_2 = \{ a^k b^m c^n \mid m=n \}$ **Kontextfrei**
 - $L_1 \cap L_2 = \{ a^k b^m c^n \mid k=m=n \}$ **nicht** Kontextfrei

Inhalt

- Kontext**freie** Grammatiken
- Ableitungen, Linksableitungen
- Sprachen zu einer Grammatik
- Äquivalenz
- Chomsky-Normalform
- Wortproblem, CYK
- Pumping Lemma für CF-Sprachen
- Kontext**abhängige** Sprachen
- Allgemeine Grammatiken (Chomsky Grammatiken)
- Chomsky-Hierarchie

Kontextabhängige Grammatiken

■ Kontextfreie Regel

– $A \rightarrow \gamma$

■ Kontextabhängige Regel

– $\alpha A \beta \rightarrow \alpha \gamma \beta$

– Kommt ein A in einem bestimmten Kontext (α, β) vor, so darf es zu γ reduziert werden

Beispiel

K_{anbn}

(1) $S \rightarrow aSBC$

(2) $\quad \quad \quad | \varepsilon$

(3) $CB \rightarrow BC$

(4) $aB \rightarrow ab$

(5) $bB \rightarrow bb$

(6) $bC \rightarrow bc$

(7) $cC \rightarrow cc$

Eigentlich **nicht** kontextabhängig, lässt sich aber ersetzen durch:

$CB \rightarrow XB$
 $XB \rightarrow XC$
 $XC \rightarrow BC$

Herleitung:

$S \Rightarrow^* a^n S(BC)^n$
 $\Rightarrow^* a^n (BC)^n$
 $\Rightarrow^* a^n B^n C^n$
 $\Rightarrow^* a^n b^n C^n$
 $\Rightarrow^* a^n b^n c^n$

(1), mehrfach
 (2), einmal
 (3), mehrfach
 (4), (5), mehrfach
 (6), (7), mehrfach

Kontextabhängige Grammatiken

■ Kontextfreie Regel

– $A \rightarrow \gamma$

■ Kontextabhängige Regel

– $\alpha A \beta \rightarrow \alpha \gamma \beta$

– Kommt ein A in einem bestimmten Kontext (α, β) vor, so darf es zu γ reduziert werden

Beispiel

K_{anbn}

(1) $S \rightarrow aSBC$

(2) $\boxed{ \mid \varepsilon}$

(3) $CB \rightarrow BC$

(4) $aB \rightarrow ab$

(5) $bB \rightarrow bb$

(6) $bC \rightarrow bc$

(7) $cC \rightarrow cc$

Eigentlich **nicht** erlaubt,
lässt sich aber
ersetzen durch:

Herleitung:

$S \Rightarrow^* a^n S(BC)^n$
 $\Rightarrow^* a^n (BC)^n$
 $\Rightarrow^* a^n B^n C^n$
 $\Rightarrow^* a^n b^n C^n$
 $\Rightarrow^* a^n b^n c^n$

(1), mehrfach
 (2), einmal
 (3), mehrfach
 (4), (5), mehrfach
 (6), (7), mehrfach

$S \rightarrow S' \mid \varepsilon$
 $S' \rightarrow aS'BC \mid aBC$

Allgemeine Grammatiken

- Eine allgemeine Grammatik ist ein Tupel

- $G = (V, T, P, S)$ mit
 - $S \in V$ und $P \subseteq (V \cup T)^+ \times (V \cup T)^*$
 - wir schreiben $(\sigma \rightarrow \tau)$ für $(\sigma, \tau) \in P$

Vergl. kontextfreie Grammatik:
Quadrupel $G = (V, T, P, S)$
mit $T \cap V = \emptyset$, $S \in V$ und
 $P \subseteq V \times (T \cup V)^*$

- Ableitung und Sprache definiert wie bisher:
 - $u \Rightarrow v$ falls $u = \alpha \sigma \gamma$, $v = \alpha \tau \gamma$, und $(\sigma \rightarrow \tau) \in P$
 - $\Rightarrow^*$ reflexiv, transitive Hülle von $\Rightarrow$

$$L(G) = \{ w \in T^* \mid S \Rightarrow^* w \}$$

Hinweis: Wir können auch
 $T := \Sigma$ setzen, da T keinen
Einschränkungen unterliegt.

Chomsky Typen (Chomsky-Hierarchie)

- Klassifikation anhand der Produktionen $(\sigma \rightarrow \tau) \in P$:
 - Typ-0 (allgemein)
 - Keine Bedingung. Jede Grammatik ist Typ-0
 - Typ-1 (kontextsensitiv)
 - Produktionen: $S \rightarrow \varepsilon$ oder $(\sigma \rightarrow \tau)$ mit $|\sigma| \leq |\tau|$
 - Äquiv. (o.Beweis): $S \rightarrow \varepsilon$ oder $(\alpha A \beta \rightarrow \alpha \gamma \beta)$ mit $A \in V$ und $\gamma \neq \varepsilon$
 - Typ-2 (kontextfrei)
 - Produktionen $A \rightarrow \tau$ mit $A \in V$
 - Typ-3 (regulär)
 - Produktionen $A \rightarrow \varepsilon$ oder $A \rightarrow eB$ mit $A, B \in V$ und $e \in \Sigma$.

Reguläre Sprachen, Typ-3 Grammatiken und Automaten

- Aus einem nichtdeterministischen Automaten $NFA = (Q, \Sigma, \delta, F, q_0)$ baue eine Grammatik $G = (V, \Sigma, P, S)$ mit
 - $V := Q$
 - $S := q_0$
 - $P := \{ A \rightarrow eB \mid B \in \delta(A, e) \} \cup \{ A \rightarrow \varepsilon \mid A \in F \}$ G ist offensichtlich **Typ-3** (rechtslinear).

- Aus einer regulären (Typ-3) Grammatik $G = (V, \Sigma, P, S)$ baue nichtdeterministischen Automaten $NFA = (Q, \Sigma, \delta, F, q_0)$ mit
 - $Q := V$
 - $q_0 := S$
 - $\delta(A, e) := \{ B \mid (A \rightarrow eB) \in P \}$
 - $F := \{ A \in V \mid (A \rightarrow \varepsilon) \in P \}$
- In beiden Fällen: $(A \Rightarrow eB) \Leftrightarrow (B \in \delta(A, e))$ und $(A \Rightarrow \varepsilon) \Leftrightarrow (A \in F)$

Regulär = Typ-3

- Wir hatten:

1. $(A \Rightarrow eB)$ $\Leftrightarrow (B \in \delta(A,e))$ und
2. $(A \Rightarrow \varepsilon)$ $\Leftrightarrow (A \in F)$

- Es gilt: $e_1 \dots e_n \in L(G)$

$$\begin{aligned} &\Leftrightarrow \exists A_1, \dots, A_{n+1}: S = A_1 \\ &\quad \wedge A_i \rightarrow e_i A_{i+1} \quad // \text{ Fall 1. (links)} \\ &\quad \wedge A_{n+1} \rightarrow \varepsilon \quad // \text{ Fall 2. (links)} \\ &\Leftrightarrow \exists A_1, \dots, A_{n+1}: S = A_1 \\ &\quad \wedge A_{i+1} \in \delta(A_i, e_i) \quad // \text{ Fall 1. (rechts)} \\ &\quad \wedge A_{n+1} \in F \quad // \text{ Fall 2. (rechts)} \\ &\Leftrightarrow \delta^*(S, e_1 \dots e_n) \in F \end{aligned}$$

- Also

$$L(G) = L(\text{NFA})$$

Chomsky-Sprachklassen

- Eine Sprache L ist Typ- n , falls es eine Typ- n Grammatik G gibt, mit $L = L(G)$.
 - L_i sei Menge aller Typ- i Sprachen
- Es gilt:
 - $L_3 \subset L_2 \subset L_1 \subset L_0$
- Die Inklusionen sind echt
 - $L_{C3} = \{(ab)^n \mid n \in \mathbb{N}\} \in L_3$
 - $L_{C2} = \{a^n b^n \mid n \in \mathbb{N}\} \in L_2 - L_3$
 - $L_{C1} = \{a^n b^n c^n \mid n \in \mathbb{N}\} \in L_1 - L_2$
 - $L_{C0} = \{(r_1, r_2) \mid L(r_1) = L(r_2)\} \in L_0 - L_1$
(r_1, r_2 : reg. Ausdrücke)

Hierarchie von Sprachen

■ Bisherige Hierarchie von Sprachen

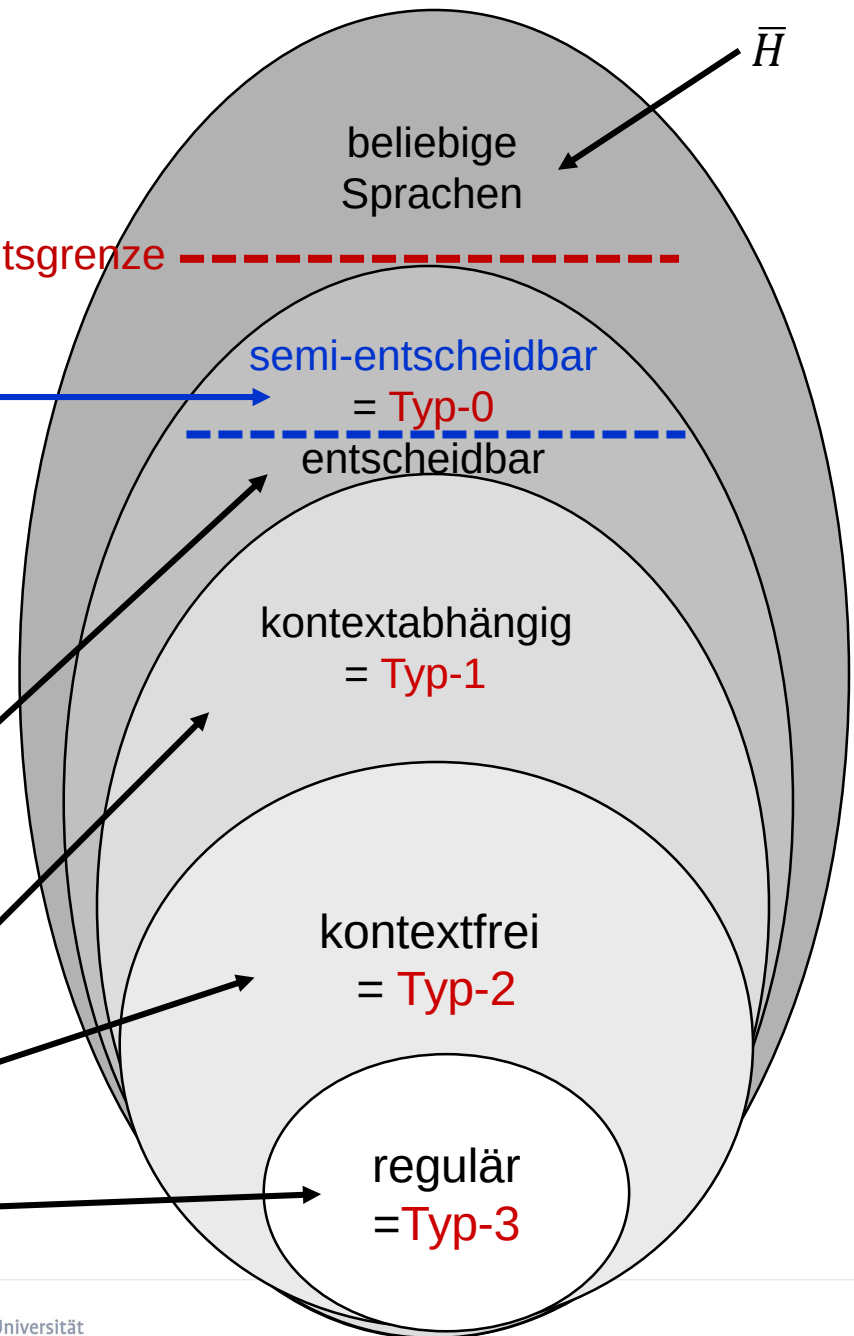
- Kann man noch verfeinern
- Insbesondere werden wir noch Sprachen kennenlernen, die **nicht entscheidbar** sind, z.B. das „Halteproblem“ H

$$L_{C0} = \{(r_1, r_2) \mid L(r_1) = L(r_2)\}$$

$$L_{C1} = \{a^n b^n c^n \mid n \in \mathbb{N}\}$$

$$L_{C2} = \{a^n b^n \mid n \in \mathbb{N}\}$$

$$L_{C3} = \{(ab)^n \mid n \in \mathbb{N}\}$$



Zusammenfassung

■ Kontextfreie Grammatiken

- Ableitungen
- $L(G)$
- Äquivalenz
- Mehrdeutigkeit

■ Chomsky-Normalform

- Entscheidbarkeit
- CYK-Algorithmus
- Abschlusseigenschaften

■ Grenzen kontextfreier Sprachen

- Pumping Lemma
- Nicht-Abschluss

■ Allgemeine Grammatiken

- Kontextabhängige Sprachen
- Typ-0, ..., Typ-3
- Regulär = Typ-3
- Chomsky-Hierarchie